UADE

TPO Programación Orientada a Objetos

Propuesta Elegida: B Clinica Veterinaria

Integrantes: Erik Berot (LU 1227491), Matias Rosental (LU 1231032), Máximo Rombolá (LU 1207165).

Profesora: Maria Angela Leon

Punto 1 - Análisis del enunciado

Actores

Actor	Tipo	Descripción
Recepcionista	Primario	Registra dueños y mascotas, asigna, reprograma y cancela turnos, y factura/anula facturas. No puede registrar diagnósticos ni tratamientos.
Veterinario	Primario	Atiende, registra consultas, diagnósticos, tratamientos y vacunas, y firma notas rectificatorias. Tiene matrícula, especialidad y horario semanal.
Administrador	Primario	Gestiona veterinarios, horarios, catálogo de vacunas y productos, y consulta reportes.
Dueño de la mascota	Secundario	No opera el sistema: solicita turnos a recepción, paga y recibe notificaciones (recordatorios, reprogramaciones).

Requerimientos Funcionales

ID	Requerimiento
RF-01	Registrar dueños (con o sin plan de salud animal).
RF-02	Registrar mascotas de distintas especies (perro, gato, ave, conejo), con datos propios de cada una.
RF-03	Transferir una mascota a otro dueño conservando su historia clínica.
RF-04	Asignar turnos evitando superposición y respetando el horario del veterinario.
RF-05	Reprogramar turnos validando la disponibilidad del profesional.
RF-06	Cancelar turnos aplicando las reglas de cargo por cancelación tardía.
RF-07	Registrar ausencias y controlar la seña tras tres ausencias seguidas.
RF-08	Atender urgencias, desplazando y reprogramando el turno siguiente si es necesario.
RF-09	Registrar consultas (motivo, peso del día, diagnóstico, observaciones, tratamientos).
RF-10	Aplicar vacunas validando edad mínima y calculando el próximo refuerzo.
RF-11	Consultar la historia clínica de las mascotas.
RF-12	Registrar notas rectificatorias firmadas por el profesional en la historia clínica.
RF-13	Gestionar el catálogo de vacunas.
RF-14	Gestionar el catálogo de productos y proveedores.
RF-15	Controlar stock por lotes (FEFO), descontar al confirmar la consulta y alertar bajo el mínimo.
RF-16	Facturar la atención (consulta + insumos) con medios de pago, descuentos y recargos.

ID	Requerimiento
RF-17	Anular facturas mediante nota de crédito.
RF-18	Emitir recordatorios de vacunas próximas a vencer.
RF-19	Emitir reportes: turnos por veterinario, productos con stock bajo, vacunas próximas a vencer.
RF-20	Gestionar veterinarios y sus horarios de atención.
RF-21	Gestionar personal de recepción.

Reglas de Negocio

Turnos

- RN-T1: la duración depende del tipo de atención: consulta general 30 min, vacunación 15 min, cirugía 120 min.
- RN-T2: cada profesional atiende solo dentro de su horario y no puede tener dos turnos superpuestos.
- RN-T3: un turno se cancela sin cargo hasta 2 horas antes; después se cobra el 20 % de la consulta.
- RN-T4: quién falta sin avisar queda como ausente; con tres ausencias seguidas debe pagar una seña para pedir nuevos turnos.
- RN-T5: una urgencia toma el primer horario del profesional de guardia; si está ocupado, desplaza al turno siguiente, que se reprograma y se notifica al dueño.

Mascotas e historia clínica

- RN-M1: toda mascota tiene un dueño responsable; un dueño puede tener varias. Si una mascota se transfiere a otro dueño, conserva su historia clínica.
- RN-M2: cada especie tiene datos propios y su propio calendario de vacunación.
- RN-M3: solo un veterinario puede registrar diagnósticos y tratamientos.
- RN-M4: la historia clínica solo admite nuevos registros; los errores se corrigen con una nota rectificatoria firmada por el profesional.
- RN-M5: no se puede indicar un medicamento si en la consulta no se registró el peso del animal.

Vacunas

- RN-V1: cada vacuna aplicada se registra con lote y fecha; el sistema calcula el próximo refuerzo.
- RN-V2: se emite recordatorio 7 días antes del vencimiento del refuerzo; pasados 30 días sin aplicarlo, la vacuna se considera vencida.
- RN-V3: no se aplica una vacuna a un animal menor a la edad mínima de esa vacuna.

Stock

- RN-S1: cada producto tiene stock mínimo; al llegar a ese valor se genera alerta de reposición.

- RN-S2: no se aplica ni vende un insumo sin stock; el stock se descuenta al confirmar la consulta, no al indicar el tratamiento.
- RN-S3: se consume primero el lote que vence antes (FEFO); un lote vencido no puede usarse.

Facturación

- RN-F1: la factura incluye la consulta y los insumos utilizados.
- RN-F2: efectivo 10 % de descuento; débito sin recargo; crédito en cuotas 8 % de recargo.
- RN-F3: los clientes con plan de salud animal tienen un 15 % de descuento; no se acumula con el de efectivo, se aplica el mayor.
- RN-F4: una factura emitida no se modifica; un error se anula con nota de crédito.

Partes Ambiguas

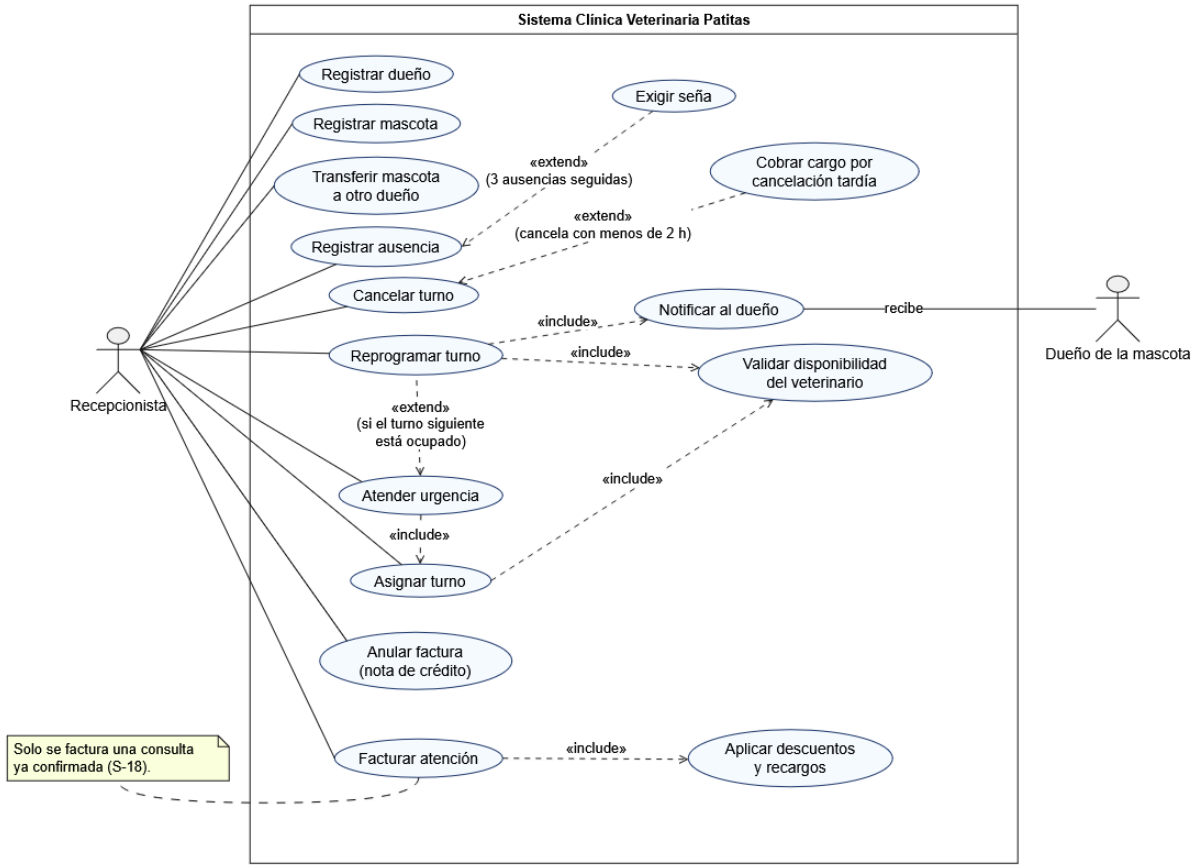
ID	Supuesto
S-01	El dueño no accede al sistema; toda gestión la hace el personal.
S-02	Las notificaciones se simulan por consola sin envío real de correo.
S-03	La seña por ausencias reiteradas es un monto fijo configurable.
S-04	La seña se descuenta de la próxima factura del cliente.
S-05	El contador de ausencias vuelve a cero cuando el dueño asiste a un turno.
S-06	El cargo por cancelación tardía (20%) se calcula sobre el precio de la consulta general y se factura como una factura de una sola línea.
S-07	El profesional de guardia es un veterinario marcado específicamente como de guardia para ese día.
S-08	El turno de urgencia dura lo mismo que una consulta general (30 min).
S-09	Si al desplazar un turno se superpone con otro, el desplazamiento se propaga en cascada al primer horario libre del mismo profesional y se notifica a cada dueño afectado.
S-10	Los precios de consulta, cirugía y vacunación están definidos en un tarifario por tipo de atención.
S-11	Cada producto físico tiene un precio de venta unitario.
S-12	El conejo se modela como subclase sin atributos extra pero con su propio calendario de vacunación.
S-13	El calendario de vacunas por especie se obtiene dinámicamente filtrando el catálogo de vacunas por especie.
S-14	La edad mínima para una vacuna se expresa en semanas.

S-15	El intervalo de refuerzo de una vacuna se expresa en días.
S-16	El stock actual de un producto es la suma de la cantidad disponible de sus lotes no vencidos.
S-17	La consulta tiene dos estados: en curso y confirmada.
S-18	Al confirmar la consulta se descuenta el stock y queda inmodificable; solo en ese momento se habilita para facturar.
S-19	Con plan de salud y pago en crédito se aplica el 15 % de descuento y luego el 8% de recargo sobre ese subtotal.
S-20	Una nota de crédito anula la factura completa; no existen anulaciones parciales.
S-21	La nota rectificatoria es un registro nuevo asociado a la consulta original, con fecha, texto y firma del veterinario.
S-22	Un veterinario tiene máximo 1 bloque de horario por día de la semana (una hora de inicio y una hora de fin).
S-23	El peso actual de la mascota se actualiza automáticamente con el peso del día registrado en cada consulta.
S-24	Al transferir una mascota, solo se guarda el dueño actual y la fecha de transferencia (manteniendo intacta la historia clínica).
S-25	A efectos de manejo de inventario y facturación, las vacunas son consideradas un producto más, contando con lotes, vencimiento físico y precio unitario.
S-26	El vencimiento de 30 días sin aplicar el refuerzo corresponde exclusivamente a la vigencia de la inmunidad en la mascota, y es totalmente independiente del vencimiento físico (stock) de los lotes de vacunas.
S-27	La nota de crédito es emitida por el recepcionista
S-28	Cada proveedor se registra con nombre, teléfono y correo electrónico. Un producto tiene un único proveedor y un proveedor puede abastecer varios productos.
S-29	Los reportes de vacunas próximas a vencer se interpretará como el vencimiento del stock físico y se agrandará el foco a todos los lotes de productos próximos a vencer
S-30	El sistema no implementará ni simulará una pasarela de pago. Simplemente utiliza el medio de pago para el cálculo del monto y se indicará en la factura cuál fue utilizado

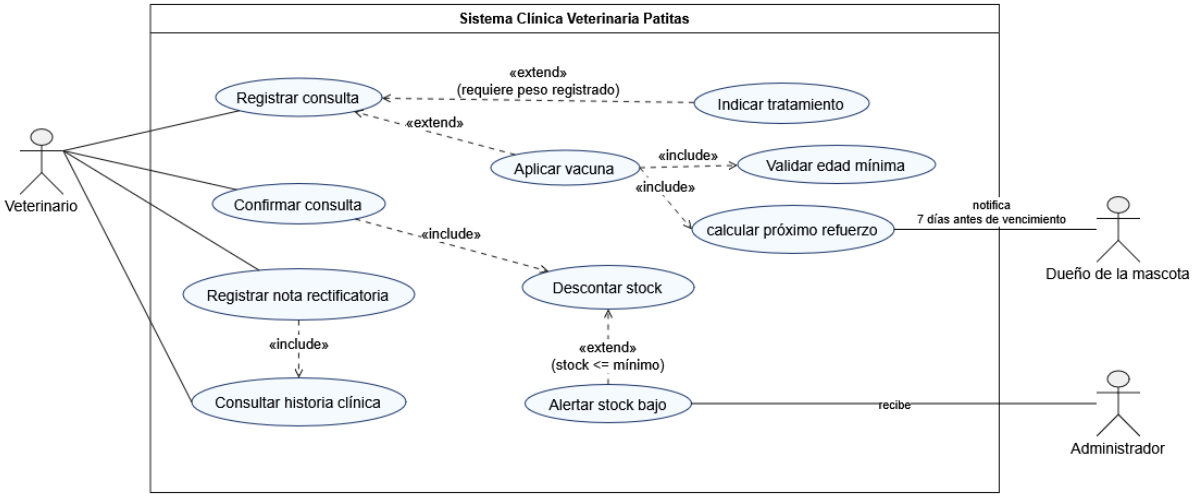
Punto 2 - Diagrama de casos de uso

Diagramas de Casos de Uso

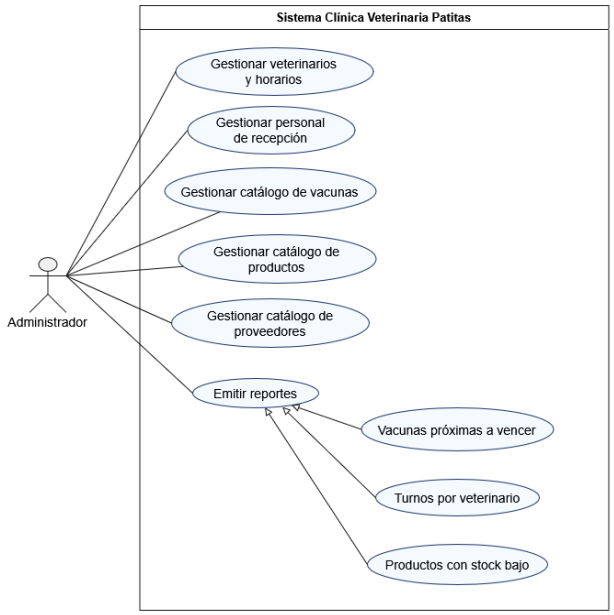
A.



B.



C.



A. Recepcionista

CU-01 Registrar dueño

- **Actor:** Recepcionista.
- **Precondición:** ninguna.
- **Flujo principal:** 1. Ingresa nombre, apellido, documento, teléfono y correo. 2. Indica si el dueño tiene plan de salud animal. 3. El sistema valida los datos y guarda el dueño.
- **Alternativos:** 3a. Documento ya registrado: el sistema informa y no crea un duplicado. 3b. Falta un dato obligatorio: el sistema rechaza el alta.

CU-02 Registrar mascota

- **Actor:** Recepcionista.
- **Precondición:** el dueño está registrado.
- **Flujo principal:** 1. Selecciona al dueño. 2. Ingresa nombre, especie, raza, sexo, fecha de nacimiento y peso actual. 3. Ingresa los datos propios de la especie (perro: chip y tamaño; gato: interior/exterior; ave: si vuela; conejo: sin extras, S-12). 4. El sistema guarda la mascota asociada al dueño.
- **Alternativos:** 1a. El dueño no existe: se ejecuta CU-01. 3a. Faltan datos propios de la especie: el sistema rechaza el alta.

CU-03 Transferir mascota a otro dueño

- **Actor:** Recepcionista.
- **Precondición:** la mascota y el nuevo dueño están registrados.
- **Flujo principal:** 1. Selecciona la mascota. 2. Selecciona el nuevo dueño. 3. El sistema guarda el dueño actual y la fecha de transferencia (S-24). 4. La historia clínica queda intacta.

- **Alternativos:** 2a. El nuevo dueño no existe: se ejecuta CU-01. 2b. El nuevo dueño es el mismo que el actual: el sistema rechaza la operación.

CU-04 Asignar turno

- **Actor:** Recepcionista (el Dueño lo solicita).
- **Precondición:** la mascota está registrada y el dueño no tiene una seña pendiente.
- **Flujo principal:** 1. El dueño solicita el turno. 2. La recepcionista elige mascota, veterinario, tipo de atención y fecha y hora. 3. El sistema asigna la duración según el tipo (30, 15 o 120 min, RN-T1). 4. Se ejecuta CU-11 (validar disponibilidad). 5. El sistema crea el turno en estado "programado".
- **Alternativos:** 4a. El veterinario no está disponible: el sistema informa y la recepcionista elige otro horario. 1a. El dueño tiene una seña requerida sin pagar: no se asigna el turno hasta registrarla (RN-T4).

CU-05 Reprogramar turno

- **Actor:** Recepcionista.
- **Precondición:** existe un turno en estado "programado".
- **Flujo principal:** 1. Selecciona el turno. 2. Indica la nueva fecha y hora (y opcionalmente otro veterinario). 3. Se ejecuta CU-11. 4. El sistema actualiza el turno. 5. Se ejecuta CU-12 para notificar al dueño.
- **Alternativos:** 3a. No hay disponibilidad: el sistema informa y se elige otro horario. 1a. El turno no está "programado" (atendido, cancelado o ausente): el sistema no permite reprogramarlo.

CU-06 Cancelar turno

- **Actor:** Recepcionista (el Dueño lo solicita).
- **Precondición:** existe un turno en estado "programado".
- **Flujo principal:** 1. El dueño pide cancelar. 2. La recepcionista selecciona el turno. 3. El sistema verifica que falten 2 horas o más (RN-T3). 4. El turno pasa a "cancelado", sin cargo.
- **Alternativos:** 3a. Faltan menos de 2 horas: se extiende con CU-13 (cargo por cancelación tardía) y el turno pasa a "cancelado".

CU-07 Registrar ausencia

- **Actor:** Recepcionista.
- **Precondición:** el turno estaba "programado" y el dueño no se presentó ni avisó.
- **Flujo principal:** 1. Selecciona el turno. 2. Lo marca como "ausente". 3. El sistema suma 1 al contador de ausencias seguidas del dueño.
- **Alternativos:** 3a. El contador llega a 3: se extiende con CU-14 (exigir seña). 3b. Cuando el dueño asiste a un turno, el contador vuelve a cero (S-05).

CU-08 Atender urgencia

- **Actor:** Recepcionista.
- **Precondición:** hay un veterinario marcado de guardia para ese día (S-07).

- **Flujo principal:** 1. Registra la urgencia para la mascota. 2. El sistema identifica el primer horario del veterinario de guardia. 3. Se ejecuta CU-04 con el turno marcado como urgencia y duración de 30 min (S-08). 4. El sistema confirma el turno de urgencia.
- **Alternativos:** 2a. El horario está ocupado: el sistema desplaza el turno siguiente y se ejecuta CU-05 para reprogramarlo, en cascada si hace falta (S-09). Se notifica a cada dueño afectado (CU-12). 2b. No hay veterinario de guardia: el sistema informa y no registra la urgencia.

CU-09 Facturar atención

- **Actor:** Recepcionista (el Dueño paga).
- **Precondición:** la consulta está confirmada y todavía no tiene factura (S-18).
- **Flujo principal:** 1. Selecciona la consulta. 2. El sistema arma las líneas: la consulta y cada insumo con cantidad y precio. 3. Se elige el medio de pago (efectivo, débito o crédito en cuotas). 4. Se ejecuta CU-15. 5. El sistema emite la factura con número y fecha, y registra el pago.
- **Alternativos:** 1a. La consulta no está confirmada: el sistema no permite facturar. 4a. El dueño tiene una seña pendiente de descontar: se descuenta del total (S-04).

CU-10 Anular factura (nota de crédito)

- **Actor:** Recepcionista.
- **Precondición:** existe una factura emitida y no anulada.
- **Flujo principal:** 1. Selecciona la factura. 2. Indica el motivo. 3. El sistema genera una nota de crédito por el total (S-20). 4. La factura queda marcada como anulada, sin modificar su contenido (RN-F4).
- **Alternativos:** 1a. La factura ya está anulada: el sistema rechaza la operación.

CU-11 Validar disponibilidad del veterinario (incluido en CU-04 y CU-05)

- **Actor:** sistema.
- **Precondición:** se conocen veterinario, fecha y hora, y duración.
- **Flujo principal:** 1. Verifica que el horario esté dentro del bloque de atención del veterinario para ese día (S-22). 2. Verifica que no se superponga con otro turno (RN-T2). 3. Devuelve “disponible”.
- **Alternativos:** 1a. Fuera de horario: devuelve “no disponible”. 2a. Hay superposición: devuelve “no disponible”.

CU-12 Notificar al dueño (incluido en CU-05 y en el desplazamiento de CU-08)

- **Actor:** sistema (el Dueño recibe).
- **Precondición:** ocurrió un evento que debe comunicarse (por ejemplo, una reprogramación).
- **Flujo principal:** 1. El sistema arma el mensaje con el motivo y la nueva fecha. 2. Lo envía al dueño, simulado por consola o log (S-02).
- **Alternativos:** 2a. El dueño no tiene correo registrado: la notificación queda igualmente en el log.

CU-13 Cobrar cargo por cancelación tardía (extiende CU-06)

- **Actor:** sistema.

- **Precondición:** se canceló un turno con menos de 2 horas de anticipación.
- **Flujo principal:** 1. Calcula el 20 % del precio de la consulta general (RN-T3, S-06). 2. Genera una factura de una sola línea por ese importe.
- **Alternativos:** ninguno.

CU-14 Exigir seña (extiende CU-07)

- **Actor:** sistema.
- **Precondición:** el dueño acumuló 3 ausencias seguidas.
- **Flujo principal:** 1. Marca al dueño con "seña requerida". 2. El monto es el valor fijo configurable (S-03). 3. Al pagarla, se habilita pedir nuevos turnos. 4. La seña se descuenta de la próxima factura (S-04).
- **Alternativos:** 3a. La seña no se paga: el dueño no puede pedir turnos.

CU-15 Aplicar descuentos y recargos (incluido en CU-09)

- **Actor:** sistema.
- **Precondición:** hay un subtotal y un medio de pago elegido.
- **Flujo principal:** 1. Efectivo: 10 % de descuento. 2. Débito: sin ajuste. 3. Crédito en cuotas: 8 % de recargo. 4. Si el dueño tiene plan de salud: 15 % de descuento (RN-F3).
- **Alternativos:** 4a. Plan de salud y pago en efectivo: no se acumulan, se aplica el mayor. 4b. Plan de salud y crédito: primero el 15 % y después el 8 % sobre ese subtotal (S-19).

B. Veterinario

CU-16 Registrar consulta

- **Actor:** Veterinario.
- **Precondición:** existe un turno "programado" de la mascota con ese veterinario.
- **Flujo principal:** 1. Abre la consulta, que queda "en curso" (S-17). 2. Registra motivo, peso del día, diagnóstico y observaciones. 3. El sistema actualiza el peso actual de la mascota (S-23). 4. Opcionalmente indica tratamientos (CU-17) y aplica vacunas (CU-18).
- **Alternativos:** 2a. No registra el peso: la consulta se guarda, pero no se podrán indicar medicamentos (RN-M5).

CU-17 Indicar tratamiento (extiende CU-16)

- **Actor:** Veterinario.
- **Precondición:** hay una consulta en curso con el peso registrado.
- **Flujo principal:** 1. Selecciona el medicamento. 2. Ingresa dosis, frecuencia y duración. 3. El sistema verifica que haya stock, sin descontarlo todavía (RN-S2). 4. Agrega el tratamiento a la consulta.
- **Alternativos:** 1a. No hay peso registrado: el sistema rechaza (RN-M5). 3a. No hay stock del medicamento: el sistema no permite indicarlo.

CU-18 Aplicar vacuna (extiende CU-16)

- **Actor:** Veterinario.
- **Precondición:** hay una consulta en curso y la vacuna existe en el catálogo para la especie.

- **Flujo principal:** 1. Selecciona la vacuna. 2. Se ejecuta CU-19. 3. El sistema asigna el lote vigente con vencimiento más próximo. 4. Registra fecha y lote. 5. Se ejecuta CU-20.
- **Alternativos:** 2a. La mascota es menor a la edad mínima: se rechaza la aplicación. 3a. No hay lotes vigentes: no se puede aplicar.

CU-19 Validar edad mínima (incluido en CU-18)

- **Actor:** sistema.
- **Precondición:** se seleccionó una vacuna para una mascota.
- **Flujo principal:** 1. Calcula la edad de la mascota en semanas a partir de su fecha de nacimiento (S-14). 2. La compara con la edad mínima de la vacuna. 3. Devuelve “válida”.
- **Alternativos:** 2a. Edad menor a la mínima: devuelve “no válida” (RN-V3).

CU-20 Calcular próximo refuerzo (incluido en CU-18)

- **Actor:** sistema (el Dueño recibe el aviso).
- **Precondición:** se aplicó una vacuna.
- **Flujo principal:** 1. Suma al día de aplicación el intervalo de refuerzo en días (S-15). 2. Guarda la fecha del próximo refuerzo. 3. Cuando faltan 7 días, notifica al dueño (RN-V2, S-02).
- **Alternativos:** 3a. Pasan 30 días del vencimiento sin aplicar el refuerzo: la vacuna se considera vencida (RN-V2, S-26).

CU-21 Confirmar consulta

- **Actor:** Veterinario.
- **Precondición:** la consulta está “en curso”.
- **Flujo principal:** 1. Revisa los datos cargados. 2. Confirma la consulta. 3. Se ejecuta CU-22. 4. La consulta queda “confirmada” e inmodificable (S-18). 5. El turno pasa a “atendido” y la consulta queda habilitada para facturar.
- **Alternativos:** 3a. Falla el descuento de stock: no se confirma y el sistema informa qué insumo falta.

CU-22 Descontar stock (incluido en CU-21)

- **Actor:** sistema.
- **Precondición:** se está confirmando una consulta con insumos o vacunas.
- **Flujo principal:** 1. Por cada insumo, toma el lote no vencido con vencimiento más próximo (FEFO, RN-S3). 2. Descuenta la cantidad usada. 3. Actualiza el stock actual del producto (S-16). 4. Verifica el stock mínimo.
- **Alternativos:** 1a. El lote no alcanza: continúa con el siguiente lote. 1b. El stock total es insuficiente: cancela todo el descuento. 4a. El stock llega al mínimo o menos: se extiende con CU-23.

CU-23 Alertar stock bajo (extiende CU-22)

- **Actor:** sistema (el Administrador recibe la alerta).
- **Precondición:** el stock de un producto quedó en el mínimo o por debajo (RN-S1).
- **Flujo principal:** 1. Genera la alerta con producto, stock actual y mínimo. 2. La envía al administrador.

- **Alternativos:** 1a. Ya hay una alerta vigente para ese producto: no se duplica.

CU-24 Registrar nota rectificatoria

- **Actor:** Veterinario.
- **Precondición:** existe una consulta en la historia clínica de la mascota.
- **Flujo principal:** 1. Se ejecuta CU-25 para ubicar la consulta. 2. Redacta el texto de la nota. 3. La firma. 4. El sistema guarda la nota como registro nuevo con fecha, texto y firma, asociada a la consulta original (S-21).
- **Alternativos:** 2a. El texto está vacío: el sistema rechaza la nota. 4a. La consulta original nunca se modifica (RN-M4).

CU-25 Consultar historia clínica

- **Actor:** Veterinario.
- **Precondición:** la mascota está registrada.
- **Flujo principal:** 1. Busca la mascota. 2. El sistema muestra sus consultas en orden cronológico, con diagnósticos, tratamientos, vacunas y notas rectificatorias.
- **Alternativos:** 1a. La mascota no tiene consultas: se muestra la historia vacía. 2a. La mascota fue transferida: se muestra la historia completa, anterior y posterior.

C. Administrador

CU-26 Gestionar veterinarios y horarios

- **Actor:** Administrador.
- **Precondición:** ninguna.
- **Flujo principal:** 1. Elige alta, modificación o baja. 2. Ingresa nombre, apellido, documento, teléfono, matrícula y especialidad. 3. Define el bloque de atención (hora de inicio y fin) de cada día de la semana (S-22). 4. Opcionalmente marca al veterinario de guardia para un día (S-07). 5. El sistema guarda los cambios.
- **Alternativos:** 2a. Matrícula o documento duplicado: se rechaza. 3a. La hora de fin es anterior o igual a la de inicio: se rechaza. 1a. Baja con turnos futuros: el sistema exige reprogramarlos antes.

CU-27 Gestionar personal de recepción

- **Actor:** Administrador.
- **Precondición:** ninguna.
- **Flujo principal:** 1. Elige alta, modificación o baja. 2. Ingresa nombre, apellido, documento y teléfono. 3. El sistema guarda los cambios.
- **Alternativos:** 2a. Documento duplicado: se rechaza.

CU-28 Gestionar catálogo de vacunas

- **Actor:** Administrador.
- **Precondición:** ninguna.
- **Flujo principal:** 1. Elige alta, modificación o baja. 2. Ingresa nombre, especies a las que aplica, edad mínima en semanas e intervalo de refuerzo en días. 3. Como la vacuna es un

producto más (S-25), ingresa precio unitario y lotes con vencimiento. 4. El sistema guarda los cambios.
- **Alternativos:** 2a. Edad mínima o intervalo no positivos: se rechaza. 1a. Baja de una vacuna ya aplicada: se da de baja sin borrar el historial.

CU-29 Gestionar catálogo de productos

- **Actor:** Administrador.
- **Precondición:** ninguna.
- **Flujo principal:** 1. Elige alta, modificación o baja. 2. Ingresa código, nombre, stock mínimo y precio unitario de venta (S-11). 3. Registra los lotes recibidos, con cantidad y fecha de vencimiento. 4. Asocia el proveedor del producto. 5. El sistema guarda los cambios.
- **Alternativos:** 2a. Código duplicado: se rechaza. 3a. Lote con fecha de vencimiento pasada: se rechaza (RN-S3). 4a. El proveedor no existe: se ejecuta CU-30 para darlo de alta. 1a. Baja de un producto con stock o con consultas asociadas: se da de baja sin borrar el historial.

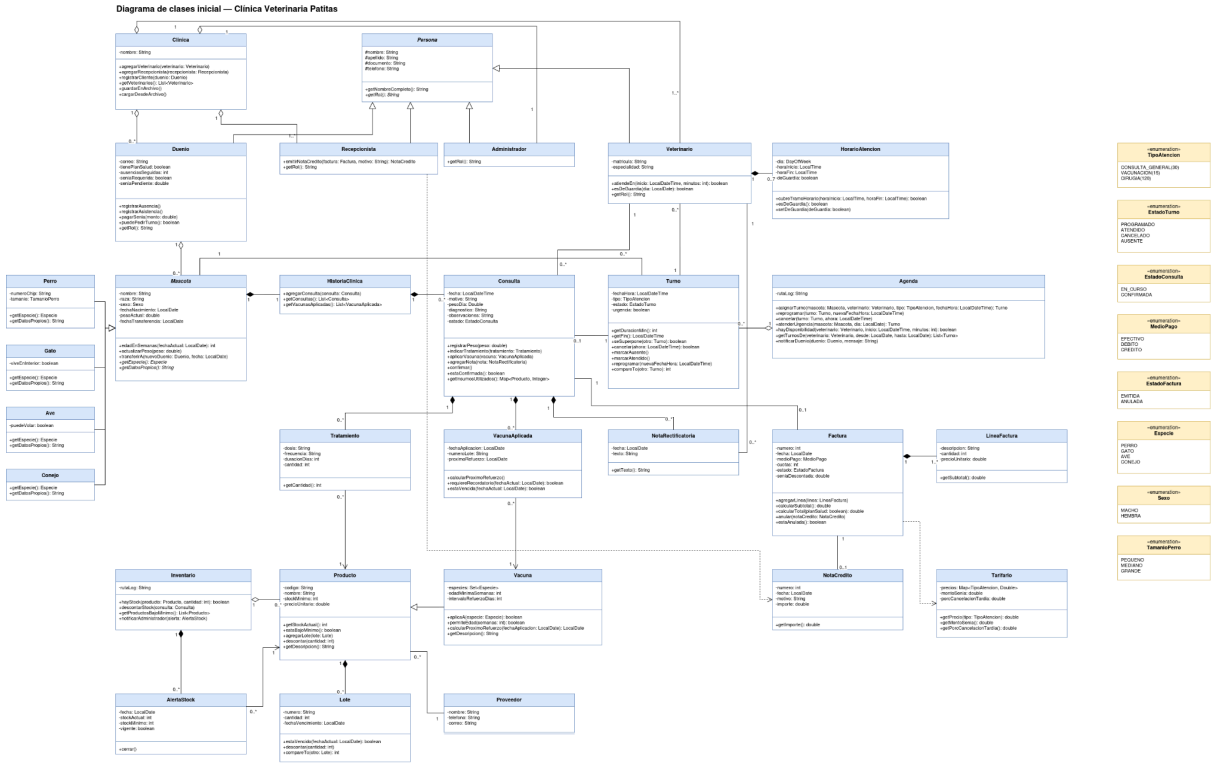
CU-30 Gestionar catálogo de proveedores

- **Actor:** Administrador.
- **Precondición:** ninguna.
- **Flujo principal:** 1. Elige alta, modificación o baja. 2. Ingresa los datos del proveedor (nombre, teléfono y correo). 3. El sistema guarda los cambios.
- **Alternativos:** 2a. Proveedor ya registrado: se rechaza el alta duplicada. 1a. Baja de un proveedor con productos asociados: el sistema no permite borrarlo hasta reasignar esos productos.

CU-31 Emitir reportes

- **Actor:** Administrador.
- **Precondición:** ninguna.
- **Flujo principal:** 1. Elige el tipo de reporte. 2. El sistema lo genera con los datos vigentes. 3. Lo muestra al administrador.
- **Alternativos:** 2a. No hay datos para el reporte: se muestra vacío.
- **Especializaciones:**
    - **Turnos por veterinario:** se selecciona un veterinario y un período, y se listan sus turnos.
    - **Productos con stock bajo:** se listan los productos con stock actual menor o igual al mínimo.
    - **Vacunas próximas a vencer:** se listan las vacunas con refuerzo próximo a vencer (RF-19).

Punto 3 - Diagrama de clases inicial



Convenciones

- **Visibilidad:** + público, - privado, # protegido.
- **Clase abstracta:** su nombre y sus métodos abstractos van en cursiva en el diagrama.
- **Composición (rombo lleno):** la parte no existe sin el todo.
- **Agregación (rombo hueco):** el todo agrupa partes que existen por sí solas.
- **Asociación:** línea simple.
- Los atributos que son colecciones o referencias a otras clases del modelo no se repiten como atributos: quedan implícitos en la relación y su multiplicidad.
- Las enumeraciones se muestran aparte y se usan como tipos de atributos.

1. Clínica y personas

Clinica

Representa a la clínica «Patitas» y agrupa a su personal y a sus clientes.

- **Atributos:** -nombre: String.
- **Métodos:**
    - agregarVeterinario(veterinario), agregarRecepcionista(recepcionista) y registrarCliente(duenio): dan de alta cada elemento en la clínica.

    - getVeterinarios(): List<Veterinario>: expone la lista, por ejemplo para elegir veterinario al asignar un turno, para el reporte de turnos por veterinario o para buscar al de guardia.
    - guardarEnArchivo() y cargarDesdeArchivo(): guardan y recuperan los datos de la clínica en archivos para que se conserven entre ejecuciones.
- **Relaciones (todas agregaciones, con Clinica en el extremo del rombo):** 1 a 1..* con Veterinario, 1 a 1..* con Recepcionista, 1 a 1 con Administrador y 1 a 0..* con Duenio (clientes).

Persona (abstracta)

Generaliza los datos comunes de todas las personas del sistema.

- **Atributos (protegidos, para que las subclases los usen):** #nombre, #apellido, #documento, #telefono, todos String.
- **Métodos:**
    - getNombreCompleto(): String: une nombre y apellido; lo heredan todas las subclases.
    - getRol(): String (abstracto): cada subclase lo sobreescribe e informa qué tipo de persona es.
- **Relaciones:** es la superclase de Duenio, Veterinario, Recepcionista y Administrador.

Duenio

Cliente de la clínica, responsable de una o más mascotas.

- **Atributos:**
    - -correo: String.
    - -tienePlanSalud: boolean: da derecho al 15 % de descuento (RN-F3).
    - -ausenciasSeguidas: int: contador de ausencias; vuelve a cero cuando asiste a un turno (S-05).
    - -seniaRequerida: boolean: se activa con la tercera ausencia seguida (RN-T4).
    - -seniaPendiente: double: monto de seña a descontar de la próxima factura (S-04).
- **Métodos:**
    - registrarAusencia(): suma una ausencia y, al llegar a 3, exige seña.
    - registrarAsistencia(): reinicia el contador de ausencias.
    - pagarSenia(monto): registra el pago de la seña y lo habilita a pedir turnos.
    - puedePedirTurno(): boolean: false si tiene seña requerida sin pagar. Lo consulta Agenda al asignar un turno.
    - getRol(): sobreescribe el de Persona.
- **Relaciones:** hereda de Persona; agregación 1 a 0..* con Mascota (es el dueño actual); forma parte de Clinica como cliente.

Recepcionista

Personal que registra dueños y mascotas, gestiona turnos y factura.

- **Atributos:** ninguno propio.
- **Métodos:**
    - emitirNotaCredito(factura, motivo): NotaCredito: crea la nota de crédito por el total de la factura y la anula (CU-10, S-27).
    - getRol().
- **Relaciones:** hereda de Persona; asociación con NotaCredito (una recepcionista emite 0..* notas; cada nota la emite una recepcionista); parte de Clinica.

Administrador

Gestiona veterinarios, horarios, catálogos y consulta reportes. Recibe las alertas de stock bajo.

- **Atributos:** ninguno propio.
- **Métodos:** getRol().
- **Relaciones:** hereda de Persona; parte de Clinica (una clínica tiene un administrador).

Veterinario

Profesional que atiende, registra consultas, diagnósticos, tratamientos y vacunas, y firma notas rectificatorias (RN-M3, RN-M4).

- **Atributos:** -matricula: String, -especialidad: String. Sus horarios y guardias no son atributos, sino que se expresan en la relación con HorarioAtencion.
- **Métodos:**
    - atiendeEn(inicio, minutos): boolean: indica si el veterinario atiende en ese tramo; busca el HorarioAtencion del día y le pregunta si lo cubre (RN-T2, CU-11).
    - esDeGuardia(dia): boolean: indica si está de guardia ese día (S-07), según su HorarioAtencion.
    - getRol().
- **Relaciones:** hereda de Persona; composición 1 a 0..7 con HorarioAtencion; asociación 0..* a 1 con Turno, con Consulta y con NotaRectificatoria; parte de Clinica.

HorarioAtencion

Bloque de atención de un veterinario para un día de la semana. Hay como máximo un bloque por día (S-22).

- **Atributos:** -dia: DayOfWeek, -horaInicio: LocalTime, -horaFin: LocalTime, -deGuardia: boolean (si ese día el veterinario es el de guardia).
- **Métodos:**
    - cubreTramoHorario(horaInicio, horaFin): boolean: true si el tramo entra completo dentro del bloque (this.horaInicio <= horaInicio y horaFin <= this.horaFin).
    - esDeGuardia(): boolean y setDeGuardia(deGuardia): consulta y cambia la marca de guardia.
- **Relaciones:** composición con Veterinario: no existe sin él.

2. Mascotas y especies

Mascota (abstracta)

Paciente de la clínica. Cada especie agrega datos propios y su propio calendario de vacunación (RN-M2).

- **Atributos:** -nombre, -raza (String), -sexo: Sexo, -fechaNacimiento: LocalDate, -pesoActual: double, -fechaTransferencia: LocalDate.
- **Métodos:**
  - edadEnSemanas(fechaActual): int: se usa para validar la edad mínima de una vacuna (RN-V3, S-14).
  - actualizarPeso(peso): el peso actual se actualiza con el peso del día de cada consulta (S-23).
  - transferirA(nuevoDuenio, fecha): cambia de dueño y guarda la fecha de transferencia; la historia clínica queda intacta (RN-M1, S-24).
  - getEspecie(): Especie y getDatosPropios(): String (abstractos): los sobreescribe cada especie.
- **Relaciones:** es la superclase de Perro, Gato, Ave y Conejo; asociada a 1 Duenio (el actual) y 0..* Turno; composición 1 a 1 con HistoriaClinica.

Perro, Gato, Ave y Conejo

Subclases concretas de Mascota. Todas sobreescriben getEspecie() (devuelve su valor de Especie) y getDatosPropios() (devuelve sus datos específicos como texto).

- **Perro:** -numeroChip: String, -tamanio: TamanioPerro.
- **Gato:** -viveEnInterior: boolean.
- **Ave:** -puedeVolar: boolean.
- **Conejo:** sin atributos extra (S-12); igual tiene su propio calendario de vacunación, que se obtiene filtrando el catálogo por especie (S-13).

HistoriaClinica

Conjunto de consultas de una mascota. Solo crece: nada se borra ni se modifica (RN-M4).

- **Atributos:** ninguno propio; las consultas son la relación.
- **Métodos:**
  - agregarConsulta(consulta): único modo de incorporar registros.
  - getConsultas(): List<Consulta>: devuelve la lista en orden cronológico, de solo lectura (CU-25).
  - getVacunasAplicadas(): List<VacunaAplicada>: reúne las vacunas de todas las consultas, para el calendario y los recordatorios.
- **Relaciones:** composición 1 a 1 con Mascota; composición 1 a 0..* con Consulta.

3. Turnos y agenda

Turno

Reserva de una mascota con un veterinario para una fecha y hora, según un tipo de atención. Puede ser de urgencia.

- **Atributos:** -fechaHora: LocalDateTime, -tipo: TipoAtencion (define la duración, RN-T1), -estado: EstadoTurno, -urgencia: boolean.
- **Métodos:**
    - getDuracionMin(): int: devuelve los minutos según el tipo (30, 15 o 120).
    - getFin(): LocalDateTime: fechaHora más la duración.
    - seSuperpone(otro): boolean: detecta superposición con otro turno (RN-T2).
    - cancelar(ahora): boolean: cancela el turno; devuelve true si hubo cargo por cancelación tardía, es decir, si faltaban menos de 2 horas (RN-T3).
    - marcarAusente() y marcarAtendido(): cambian el estado.
    - reprogramar(nuevaFechaHora): cambia la fecha y hora de un turno programado.
    - compareTo(otro): int: orden natural por fecha y hora (Comparable<Turno>).
- **Relaciones:** agregado por Agenda (1 a 0..*); asociación 0..* a 1 con Mascota y con Veterinario; asociación 1 a 0..1 con Consulta (un turno atendido origina una consulta).

Agenda

Administra todos los turnos de la clínica y concentra las reglas de asignación.

- **Atributos:** -rutaLog: String, el archivo donde escribe las notificaciones.
- **Métodos:**
    - asignarTurno(mascota, veterinario, tipo, fechaHora): Turno: crea el turno si el dueño puede pedirlo y el veterinario está disponible (CU-04).
    - reprogramar(turno, nuevaFechaHora): valida disponibilidad y mueve el turno (CU-05).
    - cancelar(turno, ahora): cancela con o sin cargo (CU-06).
    - atenderUrgencia(mascota, dia): Turno: toma el primer horario del veterinario de guardia y desplaza el turno siguiente, en cascada si hace falta (CU-08, S-09).
    - hayDisponibilidad(veterinario, inicio, minutos): boolean: para validar que el turno está dentro del horario del vet y sin superposición (CU-11).
    - getTurnosDe(veterinario, desde, hasta): List<Turno>: base del reporte de turnos por veterinario.
    - notificarDuenio(duenio, mensaje), escribe en el log el aviso al dueño por reprogramaciones y desplazamientos por urgencia (CU-05, CU-08, S-02).
- **Relaciones:** agregación 1 a 0..* con Turno.

4. Atención clínica

Consulta

Registro de una atención. Tiene dos estados: en curso y confirmada (S-17). Al confirmarse queda inmodificable y habilita la facturación (S-18).

- **Atributos:** -fecha: LocalDateTime, -motivo, -diagnostico, -observaciones (String), -pesoDia: Double (puede ser nulo si no se registró), -estado: EstadoConsulta.
- **Métodos:**
    - registrarPeso(peso): guarda el peso del día; sin él no se puede indicar medicación (RN-M5).
    - indicarTratamiento(tratamiento): lo agrega si hay peso y stock (CU-17).
    - aplicarVacuna(vacuna): agrega la vacuna aplicada (CU-18).
    - agregarNota(nota): asocia una nota rectificatoria (CU-24).
    - confirmar(): pasa a confirmada; el descuento de stock lo hace Inventario.
    - estaConfirmada(): boolean.
    - getInsumosUtilizados(): Map<Producto, Integer>: insumos con cantidad, para descontar stock y armar las líneas de la factura.
- **Relaciones:** composición con HistoriaClinica; composición 1 a 0..* con Tratamiento, VacunaAplicada y NotaRectificatoria; asociación a 1 Veterinario y 0..1 Turno; asociada 0..1 a Factura.

Tratamiento

Indicación de un medicamento dentro de una consulta.

- **Atributos:** -dosis: String, -frecuencia: String, -duracionDias: int, -cantidad: int (unidades que se consumirán al confirmar).
- **Métodos:** getCantidad(): int.
- **Relaciones:** composición con Consulta; asociación hacia Producto (el medicamento).

VacunaAplicada

Cada vacuna que se aplicó a una mascota, con su lote y su próximo refuerzo (RN-V1).

- **Atributos:** -fechaAplicacion: LocalDate, -numeroLote: String, -proximoRefuerzo: LocalDate.
- **Métodos:**
    - calcularProximoRefuerzo(): suma el intervalo de la vacuna a la fecha de aplicación (S-15).
    - requiereRecordatorio(fechaActual): boolean: true desde 7 días antes del refuerzo (RN-V2).
    - estaVencida(fechaActual): boolean: true si pasaron 30 días del refuerzo sin aplicarlo (RN-V2, S-26).
- **Relaciones:** composición con Consulta; asociación hacia Vacuna.

NotaRectificatoria

Corrige un error de una consulta sin modificarla (RN-M4, S-21).

- **Atributos:** -fecha: LocalDate, -texto: String.
- **Métodos:** getTexto(): String.
- **Relaciones:** composición con Consulta; asociación 0..* a 1 con Veterinario (quien la firma).

5. Inventario y stock

Inventario

Controla el stock de medicamentos, insumos y vacunas, y concentra el descuento al confirmar una consulta.

- **Atributos:** -rutaLog: String, el archivo donde escribe las alertas.
- **Métodos:**
    - hayStock(producto, cantidad): boolean: verifica que haya existencias en lotes no vencidos (RN-S2); se usa al indicar un tratamiento, sin descontar todavía.
    - descontarStock(consulta): al confirmar la consulta, descuenta cada insumo siguiendo FEFO, es decir, del lote que vence antes (RN-S3, CU-22). Si el stock total no alcanza, no descuenta nada. Si un producto queda en su mínimo o por debajo, genera una AlertaStock sin duplicar una vigente (RN-S1, CU-23).
    - getProductosBajoMinimo(): List<Producto>: base del reporte de productos con stock bajo.
    - notificarAdministrador(alerta: AlertaStock), escribe la alerta de stock bajo en el log (CU-23).
- **Relaciones:** agregación 1 a 0..* con Producto; composición 1 a 0..* con AlertaStock.

Producto

Cualquier insumo o medicamento que la clínica compra y vende.

- **Atributos:** -codigo: String (único), -nombre: String, -stockMinimo: int, -precioUnitario: double (precio de venta, S-11).
- **Métodos:**
    - getStockActual(): int: suma las cantidades de sus lotes no vencidos (S-16).
    - estaBajoMinimo(): boolean: true si el stock actual es menor o igual al mínimo.
    - agregarLote(lote): registra un lote recibido; rechaza uno ya vencido.
    - descontar(cantidad): toma la cantidad de los lotes ordenados por vencimiento (FEFO).
    - getDescripcion(): String: texto descriptivo; lo sobreescribe Vacuna.
- **Relaciones:** composición 1 a 0..* con Lote; asociación 0..* a 1 con Proveedor; agregado por Inventario; superclase de Vacuna; referenciado por Tratamiento y por AlertaStock.

Vacuna

Entrada del catálogo de vacunas. Es un producto más (S-25): tiene lotes, vencimiento físico y precio.

- **Atributos:** -especies: Set<Especie> (a qué especies se aplica), -edadMinimaSemanas: int (S-14), -intervaloRefuerzoDias: int (S-15).
- **Métodos:**
  - aplicaA(especie): boolean: sirve para armar el calendario de cada especie (S-13).
  - permiteEdad(semanas): boolean: valida la edad mínima (RN-V3).
  - calcularProximoRefuerzo(fechaAplicacion): LocalDate: fecha de aplicación más el intervalo.
  - getDescripcion(): sobreescribe el de Producto.
- **Relaciones:** hereda de Producto; referenciada por VacunaAplicada.

Lote

Partida de un producto con su propia fecha de vencimiento.

- **Atributos:** -numero: String, -cantidad: int, -fechaVencimiento: LocalDate (vencimiento físico, independiente de la vigencia de la inmunidad, S-26).
- **Métodos:**
  - estaVencido(fechaActual): boolean: un lote vencido no puede usarse (RN-S3).
  - descontar(cantidad): resta unidades del lote.
  - compareTo(otro): int: orden natural por vencimiento, que implementa FEFO (Comparable<Lote>).
- **Relaciones:** composición con Producto.

Proveedor

Empresa que abastece productos (S-28).

- **Atributos:** -nombre, -telefono, -correo (String).
- **Métodos:** ninguno propio.
- **Relaciones:** un proveedor abastece 0..* productos y cada producto tiene un único proveedor.

AlertaStock

Aviso de reposición que recibe el administrador (RN-S1).

- **Atributos:** -fecha: LocalDate, -stockActual: int, -stockMinimo: int, -vigente: boolean (evita duplicar alertas del mismo producto).
- **Métodos:** cerrar(): la marca como no vigente cuando se repone el stock.
- **Relaciones:** composición con Inventario; asociación 0..* a 1 con Producto.

6. Facturación

Factura

Comprobante de una consulta confirmada: incluye la consulta y los insumos usados (RN-F1). Una vez emitida no se modifica (RN-F4).

- **Atributos:**
  - -numero: int, -fecha: LocalDate.
  - -medioPago: MedioPago, -cuotas: int (para crédito).
  - -estado: EstadoFactura (emitida o anulada).
  - -seniaDescontada: double: parte de la seña pendiente del dueño que se descontó del total (S-04). Queda registrada porque la factura no se puede modificar después.
- **Métodos:**
  - agregarLinea(linea): agrega una línea de detalle.
  - calcularSubtotal(): double: suma los subtotales de las líneas.
  - calcularTotal(planSalud): double: aplica descuentos y recargos según el medio de pago y el plan de salud (RN-F2, RN-F3, S-19) y resta la seña.
  - anular(notaCredito): verifica que no esté anulada y cambia el estado a anulada, sin tocar su contenido (CU-10).
  - estaAnulada(): boolean.
- **Relaciones:** composición 1 a 1..* con LineaFactura; asociación con Consulta (una consulta confirmada tiene 0..1 factura); dependencia hacia Tarifario (precios); referenciada por NotaCredito.

LineaFactura

Un renglón del detalle: la consulta o un insumo.

- **Atributos:** -descripcion: String, -cantidad: int, -precioUnitario: double.
- **Métodos:** getSubtotal(): double (cantidad × precioUnitario).
- **Relaciones:** composición con Factura.

NotaCredito

Anula una factura completa; no hay anulaciones parciales (S-20).

- **Atributos:** -numero: int, -fecha: LocalDate, -motivo: String, -importe: double (igual al total de la factura).
- **Métodos:** getImporte(): double.
- **Relaciones:** asociación 0..1 a 1 con Factura; asociación 0..* a 1 con Recepcionista (quien la emite, S-27).

Tarifario

Concentra los valores configurables del negocio (S-03, S-10).

- **Atributos:** -precios: Map<TipoAtencion, Double> (consulta general, vacunación y cirugía), -montoSenia: double (monto fijo), -porcCancelacionTardia: double (20 %, S-06).

- **Métodos:** getPrecio(tipo): double, getMontoSenia(): double y getPorcCancelacionTardia(): double.
- **Relaciones:** Factura depende de él para obtener el precio de la consulta y calcular el cargo por cancelación tardía.

7. Enumeraciones

Enumeración	Valores	Se usa en
TipoAtencion	CONSULTA_GENERAL(30), VACUNACION(15), CIRUGIA(120); entre paréntesis, la duración en minutos	Turno, Tarifario
EstadoTurno	PROGRAMADO, ATENDIDO, CANCELADO, AUSENTE	Turno
EstadoConsulta	EN_CURSO, CONFIRMADA	Consulta
MedioPago	EFECTIVO, DEBITO, CREDITO	Factura
EstadoFactura	EMITIDA, ANULADA	Factura
Especie	PERRO, GATO, AVE, CONEJO	Mascota.getEspecie(), Vacuna
Sexo	MACHO, HEMBRA	Mascota
TamanioPerro	PEQUENO, MEDIANO, GRANDE	Perro

Punto 4 - Diagramas de secuencia

Diagrama de Secuencia 1: CU-09 - Registrar consulta

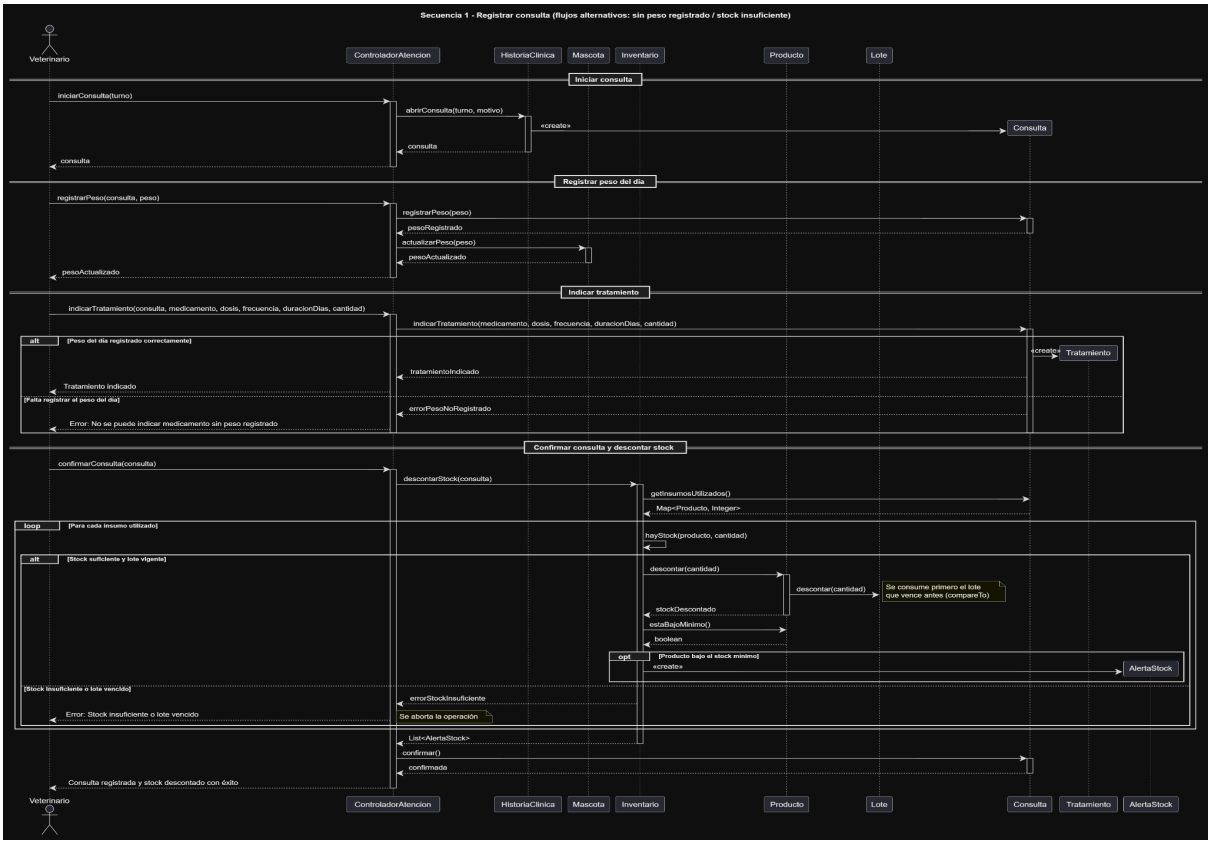
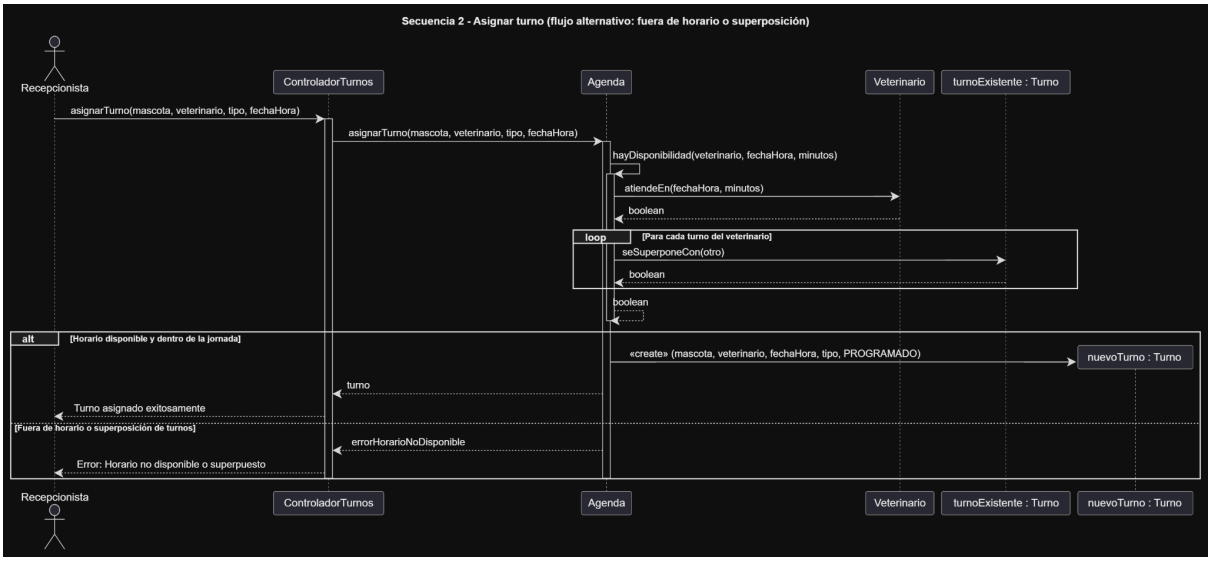


Diagrama de Secuencia 2: CU-04 - Asignar turno



Secuencia 1: Registrar consulta

El veterinario inicia la consulta desde un turno, y ControladorAtencion le pide a HistoriaClinica que abra la consulta, que es quien la crea. Después el veterinario registra el peso del día, que se guarda en la consulta y actualiza el peso de la mascota. Cuando indica un tratamiento, la consulta verifica que el peso esté registrado: si está, crea el Tratamiento, y si no, devuelve el error «No se puede indicar medicamento sin peso registrado». Al confirmar, Inventario toma los insumos usados en la consulta y, por cada uno, comprueba que haya stock y lo descuenta del lote que vence primero, avisando con una AlertaStock si el producto queda bajo el mínimo. Si falta stock o el lote está vencido, se informa el error y se aborta la operación. Si todo sale bien, se llama a Consulta.confirmar() y el veterinario recibe la confirmación.

Secuencia 2: Asignar turno

La recepcionista pide asignar un turno con la mascota, el veterinario, el tipo de atención y la fecha y hora. ControladorTurnos le pasa el pedido a Agenda, que primero revisa la disponibilidad: consulta al Veterinario si atiende en ese horario y recorre sus turnos existentes para ver si alguno se superpone con el nuevo. Si el horario está disponible y dentro de la jornada, la agenda crea el Turno en estado PROGRAMADO y la recepcionista recibe la confirmación. Si el horario cae fuera de la jornada o se superpone con otro turno, la agenda devuelve un error y la recepcionista ve «Horario no disponible o superpuesto».

Punto 5 - Aplicación de patrones

Principios SOLID Aplicados

1. Principio de Responsabilidad Única (SRP - Single Responsibility Principle)

- **Problema detectado:** Varias clases del dominio estaban sobrecargadas con responsabilidades ajenas a su propósito principal. Agenda mezclaba la asignación de turnos con la gestión de urgencias, reportes y la escritura directa de notificaciones en archivos. Inventario mezclaba control de stock con reportes y alertas físicas. Además, Clinica gestionaba el personal y los clientes, pero también se encargaba de su propia persistencia (leer y guardar archivos).
- **Decisión tomada:** Se delegaron las responsabilidades adicionales a clases específicas, purificando las clases de dominio.
    - Las urgencias pasaron a GestorUrgencias y los reportes a GeneradorReportes.
    - Las notificaciones y alertas dejaron de ser responsabilidad de Agenda e Inventario para delegarse a la interfaz Notificador (y NotificadorLog).
    - La persistencia se extrajo de Clinica hacia la clase genérica Repositorio<T> y su implementación concreta RepositorioArchivo<T>.
- **Clases en las que se ve reflejado:** Agenda, Inventario, Clinica, GestorUrgencias, GeneradorReportes, la jerarquía de Repositorio y Notificador.
- **Ventaja:** Alta cohesión y bajo acoplamiento. Cada clase cambia ahora por un único motivo. Un cambio en la forma de guardar datos (persistencia) o de enviar alertas no requiere tocar la lógica de negocio de la Clínica, la Agenda o el Inventario.

2. Principio Abierto/Cerrado (OCP - Open/Closed Principle)

- **Problema detectado:** El método Factura.calcularTotal() dependía de múltiples sentencias condicionales (if/switch) basadas en el MedioPago (que era un *enum*). Si en el futuro se quería agregar un nuevo medio de pago (ej. billeteras virtuales), era obligatorio modificar el código interno de la clase Factura.
- **Decisión tomada:** Se transformó MedioPago en una interfaz con implementaciones concretas (Efectivo, Debito, Credito), encapsulando en cada una sus propias reglas de negocio y recargos (RN-F2, RN-F3, S-19).
- **Clases en las que se ve reflejado:** Factura, Interfaz MedioPago, Efectivo, Debito, Credito.
- **Ventaja:** El diseño queda abierto a la extensión y cerrado a la modificación. Se pueden incorporar nuevos medios de pago simplemente creando una nueva clase que implemente la interfaz, sin alterar la clase Factura.

3. Principio de Inversión de Dependencias (DIP - Dependency Inversion Principle)

- **Problema detectado:** Las clases de dominio como Agenda (en notificarDuenio) e Inventario (en notificarAdministrador) escribían directamente en un archivo de log físico (dependiendo de un detalle de infraestructura mediante el atributo rutaLog). Cambiar el canal de

notificación (por ejemplo, a un correo electrónico real) obligaba a modificar el núcleo del negocio.

- **Decisión tomada:** Se extrajo la lógica de notificación hacia una abstracción. Las clases de negocio ahora dependen de la interfaz Notificador. La escritura en el log se encapsuló en la implementación concreta NotificadorLog.
- **Clases en las que se ve reflejado:** ControladorTurnos, ControladorAtencion, GestorUrgencias, ServicioRecordatorios (quienes ahora delegan), Interfaz Notificador y clase NotificadorLog.
- **Ventaja:** El dominio se desacopla de los detalles de infraestructura (salida/I-O). Permite cambiar el mecanismo de notificación en un solo lugar y facilita el testing (se pueden usar *Mocks* de la interfaz sin escribir archivos reales).

Patrones GRASP Aplicados

1. Controlador

- **Problema detectado:** Los casos de uso carecían de un punto de entrada claro desde la interfaz de usuario. Si la capa de presentación se comunicaba directamente con las clases de dominio, esta tendría que coordinar la lógica, generando un fuerte acoplamiento.
- **Decisión tomada:** Se crearon cinco controladores lógicos, agrupados por familia de casos de uso, que actúan como intermediarios. Ellos reciben los eventos del sistema, coordinan las tareas y delegan el trabajo a los objetos de dominio correspondientes.
- **Clases en las que se ve reflejado:** ControladorRegistro, ControladorTurnos, ControladorAtencion, ControladorFacturacion, ControladorAdministracion.
- **Ventaja:** Mantiene la capa de interfaz gráfica completamente desacoplada de la lógica de negocio, centralizando la coordinación del sistema.

2. Experto en Información

- **Problema detectado:** Existía una mala distribución de la información. Por ejemplo, Recepcionista.emitirNotaCredito construía la nota usando importes de facturas ajenas. A su vez, Factura.calcularTotal debía recibir el "plan de salud" como parámetro porque desconocía a su cliente, forzando a quien la llamaba a navegar por toda la cadena de relaciones (Consulta -> Turno -> Mascota -> Duenio).
- **Decisión tomada:** Se asignó la responsabilidad a la clase que posee la información necesaria. Factura ahora genera la NotaCredito (método anular) con su propio importe total. Además, se asoció directamente Factura con Duenio, permitiéndole a la factura preguntarle directamente a su cliente si posee plan de salud.
- **Clases en las que se ve reflejado:** Factura, NotaCredito, Duenio, Recepcionista.
- **Ventaja:** Se reduce el acoplamiento al evitar largas cadenas de navegación y los cálculos se encapsulan en las clases expertas que poseen los datos.

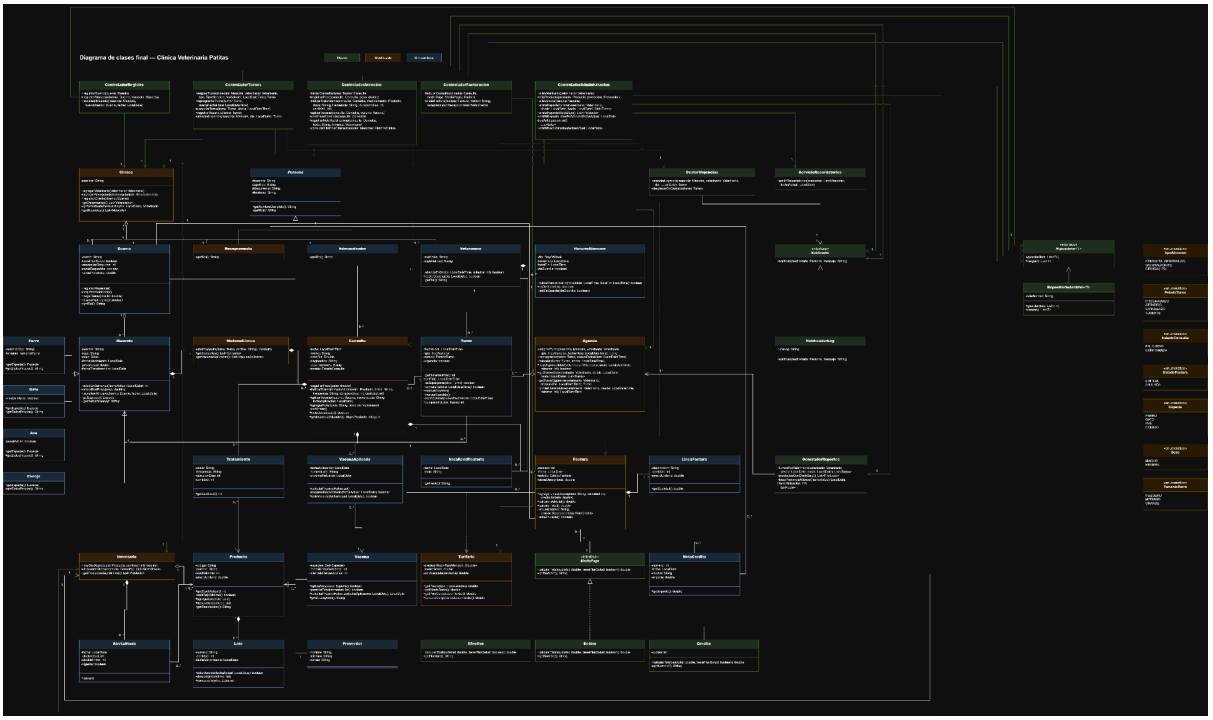
3. Creador

- **Problema detectado:** Las partes de las relaciones de composición llegaban ya instanciadas desde el exterior (ej. HistoriaClinica.agregarConsulta(consulta), Factura.agregarLinea(linea)). Esto permitía la existencia de "partes" huérfanas sin su "todo", repartía la lógica de creación en los controladores y dificultaba un punto central para las validaciones.
- **Decisión tomada:** Se aplicó el patrón Creador garantizando que el "todo" instancie a sus "partes". Ahora, los métodos reciben parámetros primitivos o de datos y ejecutan el new internamente (HistoriaClinica crea la Consulta; Consulta crea el Tratamiento, etc.).
- **Clases en las que se ve reflejado:** HistoriaClinica, Consulta, Tratamiento, VacunaAplicada, NotaRectificatoria, Factura, LineaFactura.
- **Ventaja:** Se garantiza la integridad de la composición (las partes no existen sin su todo) y asegura que las reglas de negocio (ej. validar el peso antes de un tratamiento) se ejecuten en el momento exacto de la creación.

4. Fabricación Pura

- **Problema detectado:** Clases centrales del dominio estaban sobrecargadas con lógicas que no representaban conceptos puros del negocio real, disminuyendo su cohesión. Clinica mezclaba la gestión del personal con persistencia física (guardarEnArchivo), y Agenda mezclaba los turnos con notificaciones y reportes.
- **Decisión tomada:** Se crearon clases artificiales (Fabricaciones Puras) para agrupar estos comportamientos técnicos o de coordinación que no encajan en las abstracciones del mundo real.
- **Clases en las que se ve reflejado:** GestorUrgencias, GeneradorReportes, ServicioRecordatorios, NotificadorLog y Repositorio.
- **Ventaja:** Preserva la alta cohesión y el bajo acoplamiento en las clases del modelo de dominio, separando los conceptos del negocio de las utilidades técnicas o coordinaciones complejas.

Punto 6 - Diagrama de clases final



Convenciones

- **Visibilidad:** + público, - privado, # protegido.
- **Clase abstracta o interfaz:** el nombre va en cursiva; las interfaces llevan ««interface»».
- **Composición (rombo lleno):** la parte no existe sin el todo. **Agregación (rombo hueco):** el todo agrupa partes que existen por sí solas.
- **Asociación navegable (flecha abierta):** la clase de origen guarda una referencia a la de destino. **Dependencia (flecha punteada):** la usa de forma pasajera, por ejemplo como parámetro.
- **Flechas verdes:** relaciones de los controladores y servicios con las clases que coordinan.
- Los atributos que son colecciones o referencias a otras clases del modelo no se repiten como atributos: quedan implícitos en la relación y su multiplicidad.
- Las enumeraciones se muestran aparte y se usan como tipos de atributos.

1. Qué cambió respecto del diagrama inicial

- **Controladores (nuevos):** ControladorRegistro, ControladorTurnos, ControladorAtencion, ControladorFacturacion y ControladorAdministracion. El «Sistema» de los diagramas de secuencia pasa a ser el controlador correspondiente.
- **Agenda se dividió:** las urgencias pasaron a GestorUrgencias y Agenda dejó de notificar.

- **MedioPago:** dejó de ser un enum y pasó a ser una interfaz con Efectivo, Debito y Credito.
- **Notificaciones:** Notificador (interfaz) y NotificadorLog reemplazan a Agenda.notificarDuenio e Inventario.notificarAdministrador. Inventario.descontarStock devuelve las alertas generadas para que el controlador las notifique.
- **Persistencia:** Repositorio<T> (interfaz) y RepositorioArchivo<T> reemplazan a Clinica.guardarEnArchivo y cargarDesdeArchivo.
- **Reportes y recordatorios:** GeneradorReportes (turnos por veterinario, stock bajo y lotes próximos a vencer) y ServicioRecordatorios.
- **Anulación de facturas:** Recepcionista.emitirNotaCredito desapareció; Factura.anular(motivo, emisor) genera la NotaCredito.
- **Cliente de la factura:** Factura se asocia con Duenio y calcularTotal() ya no recibe el plan de salud.
- **Creación de las partes:** HistoriaClinica.abrirConsulta, Consulta.indicarTratamiento, aplicarVacuna y agregarNota, y Factura.agregarLinea crean ellas mismas las partes de sus composiciones.
- **Otros ajustes:** Clinica suma getVeterinarioDeGuardia y getMascotas; Tarifario suma calcularCargoCancelacionTardia; Inventario.descontarStock devuelve las alertas generadas.

2. Controladores (GRASP: Controlador)

Cada controlador es el punto de entrada de una familia de casos de uso. No contiene reglas de negocio: recibe la solicitud de la interfaz, busca los objetos necesarios, delega en ellos y, si hace falta, notifica o persiste. No tienen atributos propios; las clases que coordinan son las asociaciones del diagrama.

ControladorRegistro

Casos de uso de alta de dueños y mascotas (CU-01, CU-02, CU-03).

- **Métodos:**
  - registrarDuenio(duenio): da de alta al cliente en la clínica y lo guarda.
  - registrarMascota(duenio, mascota): asocia la mascota al dueño.
  - transferirMascota(mascota, nuevoDuenio, fecha): delega en Mascota.transferirA; la historia clínica queda intacta.
- **Relaciones:** asociación navegable con Clinica y con Repositorio.

ControladorTurnos

Casos de uso de turnos y urgencias (CU-04 a CU-08).

- **Métodos:**
  - asignarTurno(mascota, veterinario, tipo, fechaHora): Turno: verifica que el dueño pueda pedir turno y delega en Agenda.
  - reprogramarTurno(turno, nuevaFechaHora) y cancelarTurno(turno, ahora): delegan en Agenda; al reprogramar, notifica al dueño.

    - registrarAusencia(turno): marca el turno como ausente y suma la ausencia al dueño (RN-T4).
    - atenderUrgencia(mascota, dia): Turno: obtiene el veterinario de guardia de Clinica y delega en GestorUrgencias.
- **Relaciones:** asociación navegable con Agenda, GestorUrgencias, Clinica, Notificador y Repositorio.

ControladorAtencion

Casos de uso de la atención clínica (CU-16 a CU-25).

- **Métodos:**
    - iniciarConsulta(turno): Consulta: abre la consulta en la historia clínica de la mascota.
    - registrarPeso, indicarTratamiento, aplicarVacuna y registrarNotaRectificatoria: reciben la consulta y los datos sueltos y delegan en Consulta.
    - confirmarConsulta(consulta): confirma la consulta, hace que Inventario descuente el stock y notifica al administrador las alertas de stock bajo que devuelva.
    - consultarHistoriaClinica(mascota): HistoriaClinica: devuelve la historia para mostrarla.
- **Relaciones:** dependencia con Consulta (la recibe por parámetro); asociación navegable con Inventario y Notificador.

ControladorFacturacion

Casos de uso de CU-09 y CU-10.

- **Métodos:**
    - facturarConsulta(consulta, medioPago): Factura: crea la factura de una consulta confirmada, con la línea de la consulta (precio del Tarifario) y una línea por insumo, y la guarda.
    - anularFactura(factura, motivo, recepcionista): NotaCredito: delega en Factura.anular.
- **Relaciones:** dependencia con Factura; asociación navegable con Tarifario y Repositorio.

ControladorAdministracion

Casos de uso del administrador (CU-26 a CU-31) y emisión de recordatorios (CU-20).

- **Métodos:**
    - altaVeterinario, altaProducto y altaVacuna: altas de personal y de catálogos, con persistencia.
    - emitirReporteTurnos, emitirReporteStockBajo y emitirReporteLotesPorVencer(fechaActual, diasAnticipacion): delegan en GeneradorReportes.
    - emitirRecordatorios(fechaActual): pasa las mascotas de Clinica a ServicioRecordatorios.

- **Relaciones:** asociación navegable con Clinica, Inventario, GeneradorReportes, ServicioRecordatorios y Repositorio.

3. Clínica y personas

Clinica

Representa a la clínica «Patitas» y agrupa a su personal y a sus clientes.

- **Atributos:** -nombre: String.
- **Métodos:**
    - agregarVeterinario, agregarRecepcionista y registrarCliente: dan de alta cada elemento.
    - getVeterinarios(): List<Veterinario>: lista del personal profesional.
    - getVeterinarioDeGuardia(dia): Veterinario: recorre sus veterinarios y devuelve el que está de guardia ese día (S-07); lo usa ControladorTurnos en las urgencias.
    - getMascotas(): List<Mascota>: reúne las mascotas de todos sus clientes, para los recordatorios.
    - No guarda ni carga archivos: de eso se encarga el repositorio (sección 8).
- **Relaciones (agregaciones, Clinica en el extremo del rombo):** 1 a 1..* con Veterinario, 1 a 1..* con Recepcionista, 1 a 1 con Administrador y 1 a 0..* con Duenio.

Persona (abstracta)

Generaliza los datos comunes de todas las personas del sistema.

- **Atributos (protegidos):** #nombre, #apellido, #documento, #telefono.
- **Métodos:** getNombreCompleto(): String y getRol(): String (abstracto; cada subclase lo sobreescribe).
- **Relaciones:** superclase de Duenio, Veterinario, Recepcionista y Administrador. También es el tipo del destinatario de Notificador.notificar.

Duenio

Cliente de la clínica, responsable de una o más mascotas. No opera el sistema (S-01).

- **Atributos:** -correo, -tienePlanSalud (15 % de descuento, RN-F3), -ausenciasSeguidas (vuelve a cero al asistir, S-05), -seniaRequerida (RN-T4) y -seniaPendiente (monto a descontar de la próxima factura, S-04).
- **Métodos:** registrarAusencia(), registrarAsistencia(), pagarSenia(monto), puedePedirTurno(): boolean (false mientras la seña requerida no se pague) y getRol().
- **Relaciones:** hereda de Persona; agregación 1 a 0..* con Mascota; es el cliente de las Factura (0..* a 1); parte de Clinica.

Recepcionista

Personal que registra dueños y mascotas, gestiona turnos y factura. No puede registrar diagnósticos ni tratamientos (RN-M3).

- **Atributos:** ninguno propio.
- **Métodos:** getRol(). La anulación de facturas la coordina ControladorFacturacion y la ejecuta Factura.
- **Relaciones:** hereda de Persona; asociación con NotaCredito (una recepcionista emite 0..* notas); parte de Clinica.

Administrador

Gestiona veterinarios, horarios, catálogos y reportes, y recibe las alertas de stock bajo.

- **Atributos:** ninguno propio. **Métodos:** getRol().
- **Relaciones:** hereda de Persona; parte de Clinica.

Veterinario

Profesional que atiende, registra consultas, diagnósticos, tratamientos y vacunas, y firma notas rectificatorias (RN-M3, RN-M4).

- **Atributos:** -matricula, -especialidad. Sus horarios y guardias están en la relación con HorarioAtencion.
- **Métodos:** atiendeEn(inicio, minutos): boolean (busca el HorarioAtencion del día y le pregunta si cubre el tramo, RN-T2), esDeGuardia(dia): boolean y getRol().
- **Relaciones:** hereda de Persona; composición 1 a 0..7 con HorarioAtencion; asociación con Turno, Consulta y NotaRectificatoria; parte de Clinica.

HorarioAtencion

Bloque de atención de un veterinario para un día de la semana (máximo uno por día, S-22).

- **Atributos:** -dia: DayOfWeek, -horaInicio, -horaFin (LocalTime) y -deGuardia: boolean.
- **Métodos:** cubreTramoHorario(horaInicio, horaFin): boolean (el tramo entra completo en el bloque), esDeGuardia() y setDeGuardia(deGuardia).
- **Relaciones:** composición con Veterinario.

4. Mascotas e historia clínica

Mascota (abstracta), Perro, Gato, Ave y Conejo

Paciente de la clínica. Cada especie agrega datos propios (RN-M2).

- **Mascota:** -nombre, -raza, -sexo: Sexo, -fechaNacimiento, -pesoActual, -fechaTransferencia. Métodos: edadEnSemanas(fechaActual) (S-14), actualizarPeso(peso) (S-23),

transferirA(nuevoDuenio, fecha) (RN-M1, S-24) y los abstractos getEspecie() y getDatosPropios().
- **Subclases:** Perro (-numeroChip, -tamanio: TamanioPerro), Gato (-viveEnInterior), Ave (-puedeVolar) y Conejo (sin extras, S-12). Todas sobreescriben getEspecie() y getDatosPropios().
- **Relaciones:** asociada a un Duenio (el actual) y a 0..* Turno; composición 1 a 1 con HistoriaClinica.

HistoriaClinica

Conjunto de consultas de una mascota; solo crece (RN-M4).

- **Atributos:** ninguno propio.
- **Métodos:**
  - abrirConsulta(turno, motivo): Consulta: **crea** la consulta y la incorpora a la historia (GRASP: Creador).
  - getConsultas(): lista de solo lectura, en orden cronológico (CU-25).
  - getVacunasAplicadas(): reúne las vacunas de todas las consultas.
- **Relaciones:** composición con Mascota y composición 1 a 0..* con Consulta.

Consulta

Registro de una atención, con estados en curso y confirmada (S-17); al confirmarse queda inmodificable (S-18).

- **Atributos:** -fecha, -motivo, -diagnostico, -observaciones, -pesoDia: Double y -estado: EstadoConsulta.
- **Métodos:**
  - registrarPeso(peso): guarda el peso del día; sin él no se pueden indicar medicamentos (RN-M5).
  - indicarTratamiento(medicamento, dosis, frecuencia, duracionDias, cantidad): verifica el peso y **crea** el Tratamiento (Creador).
  - aplicarVacuna(vacuna, numeroLote, fechaAplicacion): **crea** la VacunaAplicada y calcula su próximo refuerzo.
  - agregarNota(texto, firmante): **crea** la NotaRectificatoria.
  - confirmar(), estaConfirmada() y getInsumosUtilizados(): Map<Producto, Integer> (insumos con cantidad, tanto para descontar stock como para armar las líneas de la factura).
- **Relaciones:** composición con HistoriaClinica; composición 1 a 0..* con Tratamiento, VacunaAplicada y NotaRectificatoria; asociación con Veterinario y con Turno (0..1 a 1); Factura la factura (0..1 a 1).

Tratamiento, VacunaAplicada y NotaRectificatoria

Son partes de Consulta; las crea la propia consulta.

- **Tratamiento:** -dosis, -frecuencia, -duracionDias, -cantidad; método getCantidad(). Asociación navegable hacia Producto (el medicamento).
- **VacunaAplicada:** -fechaAplicacion, -numeroLote, -proximoRefuerzo; métodos calcularProximoRefuerzo(), requiereRecordatorio(fechaActual) (desde 7 días antes, RN-V2) y estaVencida(fechaActual) (30 días sin refuerzo, S-26). Asociación navegable hacia Vacuna.
- **NotaRectificatoria:** -fecha, -texto; método getTexto(). Asociación 0..* a 1 con Veterinario (quien la firma, S-21).

5. Turnos y agenda

Turno

Reserva de una mascota con un veterinario, para un tipo de atención.

- **Atributos:** -fechaHora, -tipo: TipoAtencion (define la duración, RN-T1), -estado: EstadoTurno, -urgencia.
- **Métodos:** getDuracionMin(), getFin(), seSuperpone(otro) (RN-T2), cancelar(ahora): boolean (true si hay cargo por cancelación tardía, RN-T3), marcarAusente(), marcarAtendido(), reprogramar(nuevaFechaHora) y compareTo(otro) (orden por fecha y hora).
- **Relaciones:** agregado por Agenda (1 a 0..*); asociación con Mascota y Veterinario; 0..1 Consulta por turno.

Agenda

Administra los turnos de la clínica y valida su asignación. No atiende urgencias ni notifica: eso lo hacen GestorUrgencias y Notificador.

- **Atributos:** ninguno propio; los turnos son la relación.
- **Métodos:** asignarTurno(mascota, veterinario, tipo, fechaHora): Turno (**crea** el Turno, CU-04), reprogramar(turno, nuevaFechaHora) (CU-05), cancelar(turno, ahora) (CU-06), hayDisponibilidad(veterinario, inicio, minutos) (CU-11), getTurnosDe(veterinario, desde, hasta), y getTurnoSiguiente(veterinario, despuesDe) y primerHorarioLibre(veterinario, desde, minutos), que usa GestorUrgencias.
- **Relaciones:** agregación 1 a 0..* con Turno.

GestorUrgencias

Resuelve la atención de urgencias (CU-08, RN-T5, S-08, S-09). Es una responsabilidad propia, distinta de la gestión de turnos (SOLID: S; GRASP: Fabricación pura).

- **Métodos:** atenderUrgencia(mascota, veterinario, dia): Turno (toma el primer horario del veterinario de guardia; si está ocupado, desplaza el turno siguiente y avisa al dueño) y -desplazarEnCascada(turno) (privado: reprograma al primer horario libre y repite si el desplazado pisa a otro).
- **Relaciones:** asociación navegable con Agenda y con Notificador.

6. Inventario y stock

Inventario

Controla el stock de medicamentos, insumos y vacunas, y concentra el descuento al confirmar una consulta. No escribe en ningún log: devuelve las alertas y el controlador las notifica.

- **Atributos:** ninguno propio; los productos son la relación.
- **Métodos:**
    - hayStock(producto, cantidad): boolean: verifica existencias en lotes no vencidos (RN-S2); se usa al indicar un tratamiento, sin descontar.
    - descontarStock(consulta): List<AlertaStock>: al confirmar la consulta, descuenta cada insumo con FEFO, es decir, del lote que vence antes (RN-S3, CU-22). Si el stock total no alcanza, no descuenta nada. **Devuelve las alertas** de stock bajo generadas (sin duplicar una vigente, RN-S1) para que el controlador las notifique.
    - getProductosBajoMinimo(): List<Producto>: base del reporte de stock bajo.
- **Relaciones:** agregación 1 a 0..* con Producto; composición 1 a 0..* con AlertaStock.

Producto, Vacuna, Lote, Proveedor y AlertaStock

- **Producto:** -codigo, -nombre, -stockMinimo, -precioUnitario (S-11). Métodos: getStockActual() (suma de lotes no vencidos, S-16), estaBajoMinimo(), agregarLote(lote), descontar(cantidad) (FEFO) y getDescripcion(). Composición 1 a 0..* con Lote; asociación 0..* a 1 con Proveedor; superclase de Vacuna.
- **Vacuna:** es un producto más (S-25). -especies: Set<Especie>, -edadMinimaSemanas, -intervaloRefuerzoDias. Métodos: aplicaA(especie) (S-13), permiteEdad(semanas) (RN-V3), calcularProximoRefuerzo(fechaAplicacion) y getDescripcion() (sobreescribe el de Producto). VacunaAplicada apunta a ella.
- **Lote:** -numero, -cantidad, -fechaVencimiento (vencimiento físico, independiente del de la inmunidad, S-26). Métodos: estaVencido(fechaActual), descontar(cantidad) y compareTo(otro) (orden por vencimiento, que implementa FEFO). Composición con Producto.
- **Proveedor:** -nombre, -telefono, -correo (S-28). Abastece a 0..* productos; cada producto tiene uno solo.
- **AlertaStock:** -fecha, -stockActual, -stockMinimo, -vigente; método cerrar() cuando se repone el stock. Composición con Inventario; asociación 0..* a 1 con Producto.

7. Facturación y medios de pago

Factura

Comprobante de una consulta confirmada: incluye la consulta y los insumos usados (RN-F1). Una vez emitida no se modifica (RN-F4).

- **Atributos:** -numero, -fecha, -estado: EstadoFactura y -seniaDescontada (parte de la seña del dueño que se descontó, S-04). El medio de pago y el cliente son relaciones, no atributos.
- **Métodos:**
    - agregarLinea(descripcion, cantidad, precioUnitario): **crea** la LineaFactura y la suma al detalle (Creador).
    - calcularSubtotal(): double: suma los subtotales de las líneas.
    - calcularTotal(): double: le pide al Duenio asociado si lo tiene, le pasa al MedioPago el subtotal y ese dato, y resta seniaDescontada (Experto, Polimorfismo).
    - anular(motivo, emisor): NotaCredito: verifica que no esté anulada, pasa a ANULADA sin tocar su contenido y **crea** la NotaCredito por el total (S-20).
    - estaAnulada(): boolean.
- **Relaciones:** composición 1 a 1..* con LineaFactura; asociación con Consulta (0..1 a 1), con Duenio (0..* a 1) y con MedioPago (0..* a 1); dependencia hacia Tarifario; NotaCredito la anula.

LineaFactura

Un renglón del detalle: la consulta o un insumo.

- **Atributos:** -descripcion, -cantidad, -precioUnitario. **Método:** getSubtotal() (cantidad × precioUnitario).
- **Relaciones:** composición con Factura.

MedioPago (interfaz)

Define cómo se ajusta un subtotal según el medio de pago y el plan de salud (SOLID: O; GRASP: Polimorfismo).

- **Métodos:** calcularTotal(subtotal, tienePlanSalud): double y getNombre(): String.
- **Implementaciones:**
    - **Efectivo:** aplica el mayor entre el 10 % de efectivo y el 15 % del plan; no se acumulan (RN-F2, RN-F3).
    - **Debito:** sin recargo; con plan de salud, 15 % de descuento.
    - **Credito** (-cuotas: int): primero el 15 % del plan, si corresponde, y después el 8 % de recargo sobre ese subtotal (S-19).

NotaCredito

Anula una factura completa; no hay anulaciones parciales (S-20).

- **Atributos:** -numero, -fecha, -motivo, -importe (el total de la factura). **Método:** getImporte().
- **Relaciones:** asociación 0..1 a 1 con Factura; asociación 0..* a 1 con Recepcionista (quien la emite, S-27).

Tarifario

Concentra los valores configurables del negocio (S-03, S-10).

- **Atributos:** -precios: Map<TipoAtencion, Double>, -montoSenia (monto fijo) y -porcCancelacionTardia (20 %).
- **Métodos:** getPrecio(tipo), getMontoSenia(), getPorcCancelacionTardia() y calcularCargoCancelacionTardia(): double (precio de la consulta general por el porcentaje, S-06; es el experto porque tiene ambos datos).
- **Relaciones:** Factura depende de él; ControladorFacturacion lo usa.

8. Servicios, notificaciones y persistencia

Son clases de fabricación pura: no representan conceptos del negocio, existen para que las clases del dominio no mezclen responsabilidades ajenas.

Notificador (interfaz) y NotificadorLog

Simulan el aviso a las personas, por consola o archivo, sin enviar correos reales (S-02). Desacoplan las reglas del negocio de la forma de notificar (SOLID: D).

- **Notificador:** notificar(destinatario: Persona, mensaje: String). El destinatario es una Persona, así sirve tanto para el dueño como para el administrador.
- **NotificadorLog:** -rutaLog: String; implementa notificar escribiendo el mensaje en el log.
- **Relaciones:** NotificadorLog implementa Notificador; lo usan ControladorTurnos, ControladorAtencion, GestorUrgencias y ServicioRecordatorios.

ServicioRecordatorios

Emite los recordatorios de vacunas próximas a vencer (RN-V2, CU-20).

- **Métodos:** emitirRecordatorios(mascotas, fechaActual): recorre las vacunas aplicadas de cada mascota y, por cada una que cumple requiereRecordatorio(fechaActual), notifica al dueño.
- **Relaciones:** asociación navegable con Notificador.

GeneradorReportes

Concentra los reportes del administrador (CU-31, RF-19).

- **Métodos:** turnosPorVeterinario(veterinario, desde, hasta), productosConStockBajo() y lotesProximosAVencer(fechaActual, diasAnticipacion): List<Lote>, que devuelve los lotes de cualquier producto (incluidas las vacunas) cuyo vencimiento físico cae dentro de los días de anticipación indicados.
- **Relaciones:** asociación navegable con Agenda (para los turnos) y con Inventario (para el stock bajo y los lotes por vencer).

Repositorio<T> (interfaz) y RepositorioArchivo<T>

Guardan y recuperan los datos en archivos, para que se conserven entre ejecuciones.

- **Repositorio<T> (interfaz):** define guardar(lista: List<T>) y cargar(): List<T>. Los controladores dependen de esta interfaz y no de la forma concreta de guardar.
- **RepositorioArchivo<T>:** implementa Repositorio<T>. Tiene -rutaArchivo: String, guarda y lee la lista en un archivo y concentra el manejo de archivos y de sus excepciones (try, catch, finally). Al ser genérico, se instancia con el tipo que se quiere persistir; por ejemplo, RepositorioArchivo<Duenio> guarda a los dueños con sus mascotas e historia clínica.
- **Relaciones:** RepositorioArchivo implementa Repositorio. Se asocian con la interfaz ControladorRegistro, ControladorTurnos, ControladorFacturacion y ControladorAdministracion.

9. Enumeraciones

Enumeración	Valores	Se usa en
TipoAtencion	CONSULTA_GENERAL(30), VACUNACION(15), CIRUGIA(120); entre paréntesis, la duración en minutos	Turno, Tarifario
EstadoTurno	PROGRAMADO, ATENDIDO, CANCELADO, AUSENTE	Turno
EstadoConsulta	EN_CURSO, CONFIRMADA	Consulta
EstadoFactura	EMITIDA, ANULADA	Factura
Especie	PERRO, GATO, AVE, CONEJO	Mascota.getEspecie(), Vacuna
Sexo	MACHO, HEMBRA	Mascota
TamanioPerro	PEQUENO, MEDIANO, GRANDE	Perro

10. Jerarquías y patrones aplicados

Jerarquías con sobreescritura:

- Persona → Duenio, Veterinario, Recepcionista, Administrador (getRol()).
- Mascota → Perro, Gato, Ave, Conejo (getEspecie(), getDatosPropios()).
- Producto → Vacuna (getDescripcion()).
- MedioPago → Efectivo, Debito, Credito (calcularTotal, getNombre).
- Notificador → NotificadorLog y Repositorio → RepositorioArchivo.

Comparable lo implementan Turno (fecha y hora) y Lote (vencimiento, para FEFO).

Principios y patrones, con el problema que resuelven:

Principio	Problema detectado	Decisión	Clases
SOLID: Responsabilidad única	Agenda reunía asignación, cancelación, urgencias y notificación; los reportes estaban repartidos	Se separan las urgencias y los reportes	GestorUrgencias, GeneradorReportes, Agenda
SOLID: Abierto/cerrado	Factura.calcularTotal necesitaba condicionales por medio de pago	Interfaz con una clase por medio de pago	MedioPago, Efectivo, Debito, Credito, Factura
SOLID: Inversión de dependencias	Agenda e Inventario escribían directamente en un log (cada una con su rutaLog)	Se depende de la interfaz Notificador	Notificador, NotificadorLog, controladores y servicios
GRASP: Controlador	Los casos de uso no tenían punto de entrada	Cinco controladores por familia de casos de uso	Controlador*
GRASP: Experto	Recepcionista.emitirNotaCredito usaba el total de una factura ajena; Factura.calcularTotal(planSalud) no conocía a su cliente	Factura.anular crea la nota; Factura se asocia con Duenio	Factura, NotaCredito, Duenio, Recepcionista
GRASP: Creador	Las partes de las composiciones llegaban ya creadas desde afuera	El todo crea sus partes	HistoriaClinica, Consulta, Factura
GRASP: Fabricación pura	Urgencias, notificación, reportes y archivos estaban dentro de clases del dominio	Clases de servicio sin concepto de negocio	GestorUrgencias, GeneradorReportes, ServicioRecordatorios, NotificadorLog, RepositorioArchivo

Informe Final

Sistema implementado

Paquetes y clases: El código se organizó en `modelo` (subpaquetes `personas`, `mascotas`, `agenda`, `atencion`, `inventario`, `facturacion` con `pago`, y `enumeradores`), `controlador`, `servicio`, `persistencia` y `vistas` (`interfaces`, `consola`, `swing`); unas 7.800 líneas de Java en 70 archivos. Los puntos de entrada son `AppSwing` y `AppConsola`.

Arquitectura Modelo-Vista-Controlador (MVC): El sistema se organizó siguiendo este patrón arquitectónico, que separa la lógica de negocio de la interfaz de usuario:

Componente	Responsabilidad	Paquetes y clases
Modelo	Entidades y reglas de negocio (turnos, historia clínica, stock FEFO, facturación, etc.); no conoce ninguna vista. Incluye los servicios de dominio y la persistencia.	`modelo.*`, `servicio`, `persistencia`
Vista	Presenta los datos y recoge las entradas del usuario. Hay una interfaz por rol con dos implementaciones intercambiables: consola y Swing. No contiene reglas de negocio.	`vistas.interfaces`, `vistas.consola`, `vistas.swing`
Controlador	Recibe las acciones de la vista, valida los parámetros, delega en el modelo, persiste el estado y devuelve el resultado. Uno por familia de casos de uso.	`ControladorRegistro`, `ControladorTurnos`, `ControladorAtencion`, `ControladorFacturacion`, `ControladorAdministracion`

El flujo es siempre Vista → Controlador → Modelo, y el resultado vuelve por el mismo camino. Como las vistas solo dependen de los controladores (que reciben identificadores y documentos, no objetos del dominio), se pudo ofrecer dos interfaces sobre la misma lógica sin duplicarla, y las pruebas funcionales ejercitan los controladores directamente. Este esquema se corresponde con el patrón GRASP Controlador.

Persistencia: `EstadoSistema` agrupa clínica, agenda, inventario, facturas y tarifario; `RepositorioArchivo` lo serializa a `vetgo.dat` tras cada operación exitosa.

Cambios respecto del diseño original

Aspecto	Diseño original (Etapa 1)	Sistema final	Motivo
Persistencia	`Repositorio<T>` genérico y `RepositorioArchivo<T>` (una lista por tipo)	`Repositorio` no genérico que guarda y carga un único `EstadoSistema` con todo el grafo	Simplificar el guardado de objetos y evitar referencias rotas entre archivos

Aspecto	Diseño original (Etapa 1)	Sistema final	Motivo
Notificación	`NotificadorLog` (escribe en archivo)	`NotificadorConsola` (imprime por consola)	S-02 admite consola o log; la consola es más visible para la demostración
Tipos de atención	Consulta general, vacunación, cirugía; `Turno.urgencia: boolean`	Se agrega `TipoAtencion.URGENCIA` (30 min, con precio en el tarifario) y se elimina el booleano	Un solo atributo expresa el tipo y la duración, y la urgencia se factura con su propia tarifa
Estados	`EstadoTurno` sin «en curso»; `EstadoConsulta`: en curso / confirmada	Se agregan `EstadoTurno.EN_CURSO` y `EstadoConsulta.FACTURADA`	El turno refleja la consulta abierta; la consulta facturada no puede facturarse dos veces y deja de listarse como pendiente
Controladores	Recibían objetos del dominio	Reciben identificadores (`String`): id de mascota/turno/consulta, documento de dueño/veterinario	Desacoplar totalmente las vistas del modelo
Vistas	No estaban modeladas	Interfaces por rol con implementación de consola y Swing, y ventana de selección de rol	Adaptar el TP a la arquitectura MVC vista en clase y dar una forma al usuario de interactuar con el sistema
Facturación	`Factura.agregarLinea` crea las líneas; `calcularTotal()` público	Las líneas se arman en `ControladorFacturacion` y se pasan al constructor; el total se calcula al emitir y queda guardado; segundo constructor para facturas sin consulta (cargo por cancelación, seña)	La factura debe ser inmutable desde su emisión (RN-F4). El anterior diseño no lo aseguraba
Anulación	`Factura.anular` devuelve la `NotaCredito`	`anular(motivo, emisor)` deja la nota asociada a la factura	La nota pertenece a la factura anulada
Seña	Atributos y reglas en `Duenio`	Igual, más `ControladorFacturacion.cobrarSenia` que emite la factura de la seña; `pagarSenia` también reinicia el contador de ausencias; asistir a una consulta también lo reinicia	Cerrar el ciclo completo de señas
Urgencias	`atenderUrgencia(mascota, veterinario, día)`: primer horario del veterinario de guardia	`atenderUrgencia(mascota, veterinario)`: comienza ahora o cuando termina el turno en curso; desplaza en cascada y, si el turno desplazado no entra en	Es la interpretación más realista de «urgencia» en un sistema en uso

Aspecto	Diseño original (Etapa 1)	Sistema final	Motivo
		la jornada, lo pasa a la próxima apertura (`Veterinario.proximaApertura`)	
Vacuna aplicada	`aplicarVacuna(vacuna, numeroLote, fecha)`	`aplicarVacuna(vacuna)`: toma el primer lote vigente (el que vence antes) y la fecha de hoy	Evitar que el veterinario cargue el lote a mano y garantizar FEFO
Identificadores	`numero: int` en factura y nota	UUID en mascotas, turnos, consultas, facturas y notas; el proveedor se identifica por CUIT	Evitar contadores globales que no sobreviven a la persistencia y asegurar que cada clase del sistema se pueda identificar de forma única
Administrador	Uno por clínica (1 a 1)	Lista de administradores; todos reciben las alertas de stock	Más flexible; sin impacto en las reglas
ABM	Alta de personal y catálogos	Alta, baja y modificación en `ControladorAdministracion`, con validaciones (documento/matrícula duplicados, un bloque horario por día, lotes no vencidos, proveedor con productos no se borra, baja de veterinario solo sin turnos futuros)	Cumplir CU-26 a CU-30 completos

Pruebas y resultados

Se escribieron **39 pruebas unitarias** y **19 casos funcionales** que recorren los casos de uso principales a través de los controladores (detalle en `docs/plan-pruebas.pdf`).

Ejecución	Pasan	Fallan
Código entregado (commit `8bf79b3`, solo con D-01 resuelto para poder compilar)	54 de 58	4 (`U-27`, `U-32`, `U-33`, `PF-13`: defectos D-02 y D-03)
Con correcciones aplicadas	58 de 58	0

Casos funcionales cubiertos: registro de dueños y mascotas, transferencia, asignación (válida, superpuesta, fuera de horario, con seña pendiente), reprogramación, cancelación con y sin cargo, ausencias y seña, urgencias con desplazamiento, consulta completa (peso, tratamiento, stock, confirmación, alerta), vacunas (especie, edad, refuerzo, recordatorio), notas rectificatorias, facturación (plan de salud, efectivo, crédito), anulación, ABM de administración, reportes y persistencia.

Alcance y límites. Las pruebas automáticas no incorporan las ventanas Swing.

Dificultades encontradas

- **Pasar del diseño al código:** Varias decisiones del diagrama final resultaron poco prácticas al implementarlas y hubo que rediseñarlas.
- **Persistir los objetos:** La serialización de dueños, mascotas, historias clínicas, turnos e inventario con referencias cruzadas obligó a simplificar el repositorio a un único `EstadoSistema`.
- **Probar código que usa la hora actual:** `Agenda`, `GestorUrgencias` y `Producto` leen la fecha del sistema, lo que dificultó las pruebas.
- **Trabajo en equipo y comunicación:** La distribución del trabajo fue más difícil de lo esperado, ya que no siempre hubo una participación equilibrada ni un seguimiento adecuado de las tareas asignadas. La comunicación por WhatsApp tampoco resultó efectiva para coordinar avances y resolver dudas. Además, en algunos casos se entregaron implementaciones generadas con IA sin una revisión suficiente, lo que trasladó al resto del equipo la responsabilidad de detectar errores, corregir inconsistencias e integrar código que no siempre cumplía con lo esperado.

Distribución de tareas

Integrante	Etapa 1 (documentación y diseño)	Etapa 2 (código, pruebas, documentación)
Erik Berot	Diagrama y documento de casos de uso; documento final con todos los puntos (UML, PDF); armado de los pull requests de cada punto	Estructura del proyecto; los cinco controladores; persistencia (`Repositorio`, `EstadoSistema`); paquete `servicio`; vistas de consola y Swing, ventana principal y `Entrada`; datos de prueba; correcciones de errores; Creación y ejecución del Plan de Pruebas; `README.md` e informe final; integración de ramas (≈31 *commits*)
Matías Rosental	Análisis del enunciado (Punto 1); diagramas de secuencia	Clases del modelo, constructores y *getters*; ajustes de clases de inventario, personas y facturación (≈8 *commits*)
Máximo Rombolá	Diagrama de clases inicial y documentación; patrones aplicados	Esqueleto de clases del modelo; primeras vistas; pruebas iniciales; ajustes de controladores y servicios.(≈7 *commits*)

Reflexión del grupo

- **Diseñar antes de programar sirvió, pero el diseño no es definitivo:** Los casos de uso, las reglas y los supuestos numerados (S-01 a S-30) fueron la referencia constante, tanto para implementar como para escribir las pruebas. En cambio, varios detalles de las clases cambiaron al implementarlas, y documentar esos cambios es parte del trabajo.
- **Separar responsabilidades facilitó el cambio:** Poder tener una interfaz de consola y otra Swing sobre los mismos controladores, o cambiar la forma de persistir sin tocar el dominio, justificó los principios SOLID y los patrones GRASP aplicados en la Etapa 1.